

List of Files

```
using Printf  
include("src/printmat.jl");
```

```
files = filter(x → endswith(lowercase(x), ".ipynb"), readdir()) #all .ipynb files  
filter!(x → x != "Ch00_ListOfFiles.ipynb", files)
```

```
printmat(files)
```

```
Ch14_FX.ipynb  
Ch15_Forwards.ipynb  
Ch16_Bonds1.ipynb  
Ch17_Bonds2.ipynb  
Ch18_Bonds3.ipynb  
Ch19_Options1.ipynb  
Ch20_Options2.ipynb  
Ch21_Options3.ipynb  
Ch22_Options4.ipynb
```

FX

This notebook illustrates some basic aspects of FX pricing, for instance, the definition of currency returns and the idea of UIP. It also implements a carry trade strategy.

Load Packages and Extra Functions

```
using Printf, Dates, Statistics, JLD2  
  
include("src/printmat.jl");
```

```
using Plots  
default(size = (480,320),fmt = :png)
```

Currency Returns

There are different ways to quote exchange rates, but this notebook uses S to denote the number of domestic currency units (say, CHF if you are a Swiss investor) to buy one unit of foreign currency (say, one USD). That is, we treat foreign currency as any other asset.

The next cell calculates the return of the following simple strategy: - in $t = 0$: buy foreign currency (at the price S_0) and lend it on foreign money market (at the safe rate R_f^*). - in $t = 1$: sell the foreign currency (at the price S_1)

Since the strategy is financed by borrowing on the domestic money market (at the rate R_f), the excess return is

$$R^e = (1 + R_f^*)S_1/S_0 - (1 + R_f)$$

Notice that R_f and R_f^* are the safe rates over the investment period (for instance, one-month period). Conversion from annualized interest rates to these monthly rates is discussed under UIP (below).

A Remark on the Code

The code uses R_f^* to denote R_f^* (since there is no easy way to get a subscript f or a superscript *).

```
S0 = 1.2      #current spot FX rate, t=0
S1 = 1.25     #spot FX rate in t=1
Rf* = 0.06    #safe lending rate (foreign) between period 0 and 1
Rf = 0.04     #safe domestic borrowing rate

Re = (1+Rf*)*S1/S0 - (1+Rf)

printblue("A simple example of how to calculate the excess return from investing in a foreign currency")
xx = [S0,Rf,Rf*,S1,Re]
printmat(xx;rowNames=["S0","Rf","Rf*","S1","Currency excess return"])
```

A simple example of how to calculate the excess return from investing in a foreign currency:

S_0	1.200
R_f	0.040
R_f^*	0.060
S_1	1.250
Currency excess return	0.064

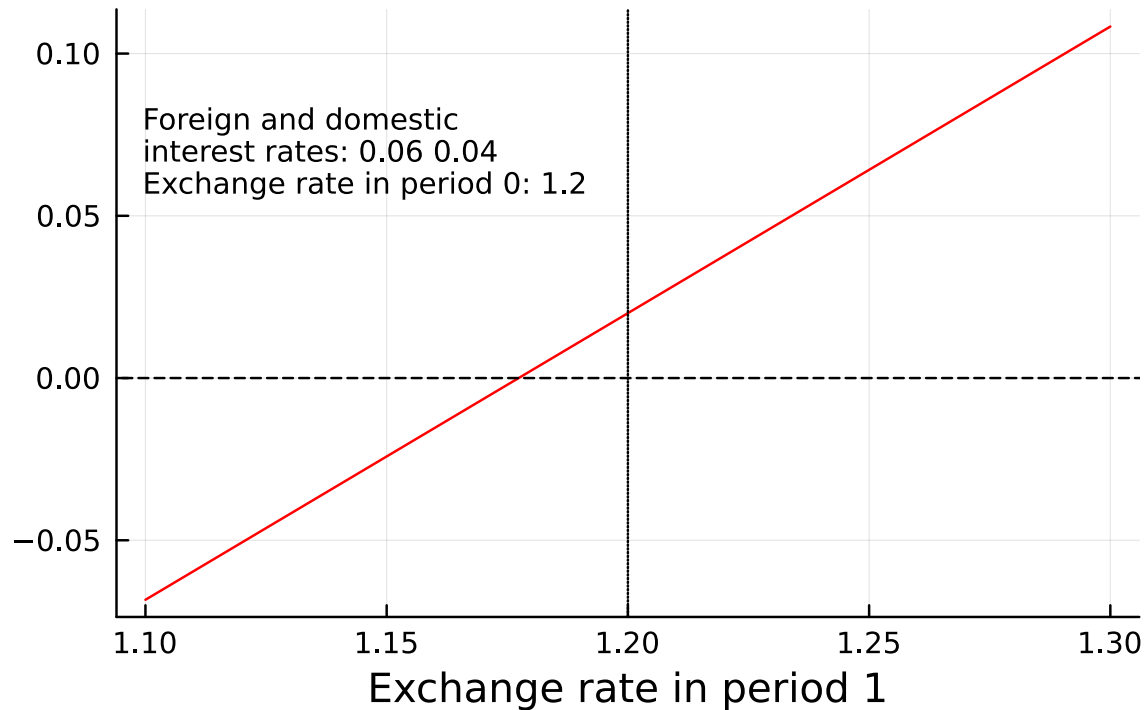
```
S1_range = 1.1:0.01:1.3      #range of different possible exchange rates in t=1

Re = (1+Rf*)*S1_range/S0 .- (1+Rf) #corresponding returns

txt = "Foreign and domestic \ninterest rates: $Rf* $Rf \nExchange rate in period 0: $S0"

p1 = plot( S1_range,Re,
           legend = nothing,
           linecolor = :red,
           title = "Currency excess return",
           xlabel = "Exchange rate in period 1",
           annotation = (1.1,0.07,text(txt,8,:left)) )
vline!([S0],linecolor=:black,line=(:dot,1))
hline!([0],linecolor=:black,line=(:dash,1))
display(p1)
```

Currency excess return



Uncovered Interest Rate Parity (UIP)

UIP *assumes* that the expected future exchange rate ($E_0 S_m$) is related to the current exchange rate and interest rates in such a way that the expected excess return of a foreign investment is zero.

In addition, interest rates are typically annualized (denoted Y and Y^* below). This means that the safe (gross) rate over an investment period of m years (eg. $m = 1/12$ for a month) is $(1 + Y)^m$.

```
S0 = 1.2          #current spot FX rate
Y  = 0.04        #annualized interest rates
Y* = 0.06
m  = 1           #investment period
ESm = S0 * (1+Y)^m / (1+Y*)^m  #implies E(excess return) = 0

printblue("Expected future exchange rate $m years ahead according to UIP:\n")
xx = [S0,Y,Y*,m,ESm]
printmat(xx;rowNames=["S0";"Y";"Y*";"m";"UIP 'expectation' of S_m"])
```

Expected future exchange rate 1 years ahead according to UIP:

S_0	1.200
Y	0.040
Y^*	0.060
m	1.000
UIP 'expectation' of S_m	1.177

A Carry Trade Strategy

Means betting on high interest currencies and against low interest rate currencies.

Load Data

The data needed is in a jld2 file.

```
(dN,Rx,fp,Rf,CurrNames) = load("Data/Fin2FxCarry2.jld2","dN","Rx","fp","Rf","CurrNames");  
(T,n) = size(Rx)
```

(372, 9)

Sorting on Forward Premia

The $m=3$ currencies with the highest forward premia (interest rate differential) in $t - 1$ are given portfolio weights $w_{CT}[t,i] = 1/m$. These are the investment currencies. The m currencies with the lowest forward premia in $t - 1$ are given portfolio weights $w_{CT}[t,i] = -1/m$. These are the funding currencies.

A Remark on the Code

- With $x = [9,7,8]$, the `rankPs(x)` function (see below) gives the output `[3,1,2]`. This says, for instance, that 7 is the lowest number (rank 1).
- `R.*w_CT` creates a $T \times n$ matrix, `sum(,dims=2)` sums across columns (for each row).

```

"""
    rankPs(x)

Calculates the ordinal rank of each element in a vector 'x'. As an alternative,
use 'ordinalrank' from the 'StatsBase.jl' package.

"""
rankPs(x) = invperm(sortperm(x))

"""
    ReturnCumPs(R,LogReturnQ=false)

Cumulate returns, with handling of NaNs (setting them to 0 in the cumulation).
Assumes net returns, but allows for log returns ('LogReturnQ=true').

"""
function ReturnCum(R,LogReturnQ=false)

    vv = isunordered.(R)
    Rb = replace(z → isunordered(z) ? 0.0 : z,R)    #do not cumulate NaNs
    P = LogReturnQ ? exp.(cumsum(Rb,dims=1)) : cumprod(1.0.+Rb,dims=1)    #P[1] = 1+R[1]
    P[vv] .= NaN    #put back the NaNs

    (n == 1) && (P = vec(P))    #to vector, needed because of P[vvb:vvb,:])

    return P
end

```

ReturnCum

```

m = 3    #number of long/short positions

(wH,wL) = [zeros(T,n) for _ in 1:2]
for t in 2:T    #loop over periods, save portfolio returns
    #local r    #local/global is needed in script
    r = rankPs(-fp[t-1,:])    #yes, it should be (-1)fp
    wL[t,:] = (r.<=m)/m    #low interest rate currencies
    wH[t,:] = ((n-m+1).<=r)/m    #high interest rate currencies
end

```

```

RL      = sum(wL.*Rx,dims=2)
RH      = sum(wH.*Rx,dims=2)
REW     = mean(Rx,dims=2)           #equally weighted
reCarry = log.(1.0.+RH) - log.(1.0.+RL)
reEW    = log.(1.0.+REW) - log.(1.0.+Rf)

```

```

xx = [reCarry reEW][2:end,:]
(μ,σ) = (mean(xx,dims=1),std(xx,dims=1))
xut = vcat(μ*12*100,σ*sqrt(12)*100,μ*sqrt(12)./σ)
printmat(xut;rowNames=["mean","std","SR"])

```

```

mean      3.501    -0.367
std       7.923     7.840
SR        0.442    -0.047

```

μ

```

1×2 Matrix{Float64}:
 0.00291762 -0.000305948

```

```

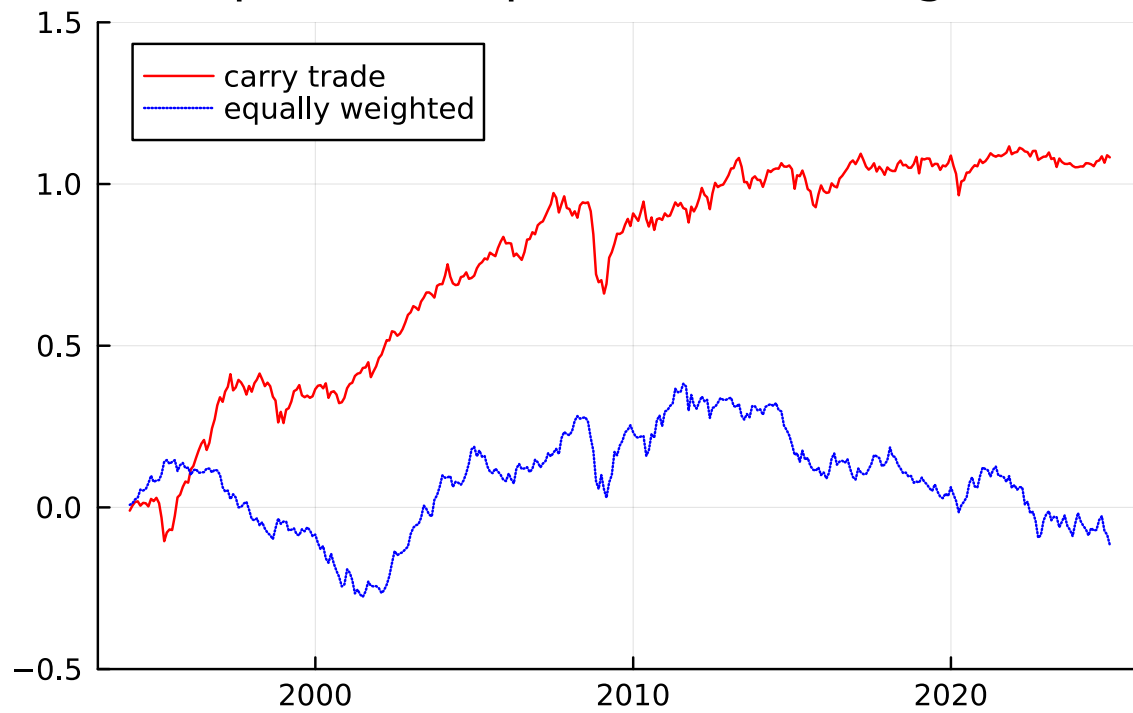
PL = ReturnCum(RL)           #cumulatin returns, for the plot
PH = ReturnCum(RH)
PEW = ReturnCum(REW)
PRf = ReturnCum(Rf)

xTicksLoc = [Date(2000),Date(2010),Date(2020)]
xTicksLab = Dates.format.(xTicksLoc,"Y")

p1 = plot( dN,hcat(log.(PH./PL),log.(PEW./PRf)),
           label = ["carry trade" "equally weighted"],
           linecolor = [:red :blue],
           linestyle = [:solid :dot],
           xticks = (xTicksLoc,xTicksLab),
           ylims = (-0.5,1.5),
           title = "Comparison of performance (log value)")
display(p1)

```

Comparison of performance (log value)



Forward Contracts

This notebook introduces forward contracts and applies the forward-spot parity to different assets.

Load Packages and Extra Functions

```
using Printf  
  
include("src/printmat.jl");
```

```
using Plots  
default(size = (480,320),fmt = :png)
```

Present Value

With a continuously compounded interest rate y , the present value of receiving Z in m years is $e^{-my}Z$.

```
(y,m,Z) = (0.05,3/4,100)  
PV = exp(-m*y)*Z  
  
printlnPs("PV of $Z when m=$m and y=$y: ",PV)
```

PV of 100 when m=0.75 and y=0.05: 96.319

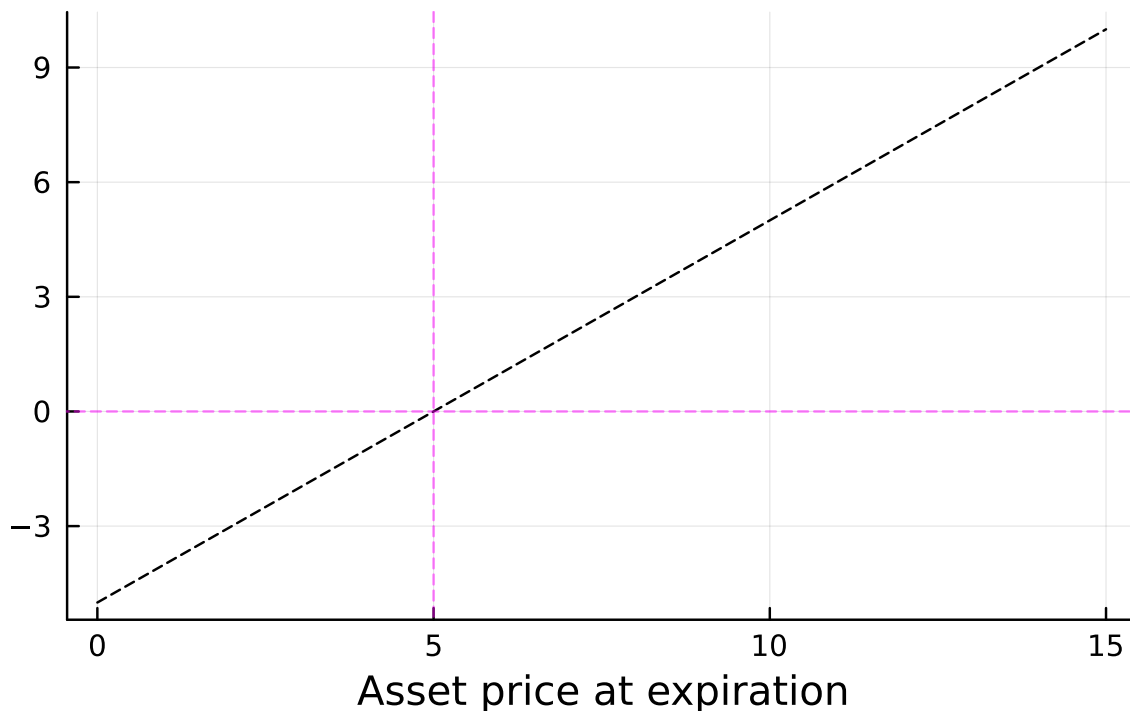
Payoff of Forward Contract

The payoff of a forward contract (at expiration, m years ahead) is $S_m - F$, where S_m is the value of the underlying (at expiration) and F is the forward price (agreed upon at inception of the contract).

```
S_m_range = range(0,15,length=16) #possible values of the underlying price at expiration
F = 5
ForwardPayoff = S_m_range .- F

p1 = plot( S_m_range,ForwardPayoff,
           linecolor = :black,
           linestyle = :dash,
           linewidth = 1,
           legend = false,
           title = "Payoff of forward contract (F = $F)",
           xlabel = "Asset price at expiration" )
hline!([0],linecolor=:magenta,line=(:dash,1,0.5))
vline!([F],linecolor=:magenta,line=(:dash,1,0.5))
#display(p1)
```

Payoff of forward contract (F = 5)



Forward-Spot Parity

For an asset without dividends (at least until expiration of the forward contract),

$$F = e^{my}S,$$

where F is the forward price (agreed upon today, to be paid m years ahead), S the *current* spot price, m the time to expiration of the forward contract and y the interest rate.

In contrast, for an asset with continuous dividends at the rate δ ,

$$F = e^{m(y-\delta)}S.$$

```
(y,m,S) = (0.05,3/4,100) #interest rate, time to expiration (in years), spot price now
F_A = exp(m*y)*S          #forward price

delta = 0.01              #dividend rate
F_B = exp(m*(y-delta))*S  #forward price when there are dividends

printblue("Forward prices:")
printmat([F_A F_B];colNames=["no dividends","with dividends"],width=16)
```

```
Forward prices:
      no dividends  with dividends
      103.821      103.045
```

Return on a Covered Strategy

Buy the underlying asset now (S) and issue a forward contract, and get the forward price (F) at expiration. This is a risk-free strategy with a gross return of

$$1 + R = F/S = e^{my},$$

if the parity holds. Thus, $\ln(1 + R)/m = y$.

```
printmat(log(F_A/S)/m,y;colNames=["ln(1+R)/m","y"])
```

```
ln(1+R)/m      y
      0.050      0.050
```

Application: Forward Price of a Bond

The forward price (in a forward contract with expiration m years ahead) of a bond that matures in $n \geq m$ is

$$F = e^{my(m)}B(n),$$

where $y(m)$ denotes the (annualized) interest rate for an m -period loan.

By definition, $1/B(m) = e^{my(m)}$. Combine to get

$$F = B(n)/B(m).$$

```
(m,n) = (5,7)           #time to expiration of forward,time to maturity of bond
(ym,yn) = (0.05,0.06)   #interest rates
Bm = exp(-m*ym)          #bond price now, maturity m
Bn = exp(-n*yn)          #bond price, maturity n
F = Bn/Bm                #forward price a bond maturing in n, delivered in m

printblue("Bond and forward prices, assuming a face value of 1:")
printmat([Bm,Bn,F];colNames=["price"],rowNames=["$(m)y-bond","$(n)y-bond","$(m)y→$(n)y forward"])
```

Bond and forward prices, assuming a face value of 1:

	price
5y-bond	0.779
7y-bond	0.657
5y->7y forward	0.844

Application: Covered Interest Rate Parity

The “dividend rate” on foreign currency is the foreign interest rate y^* (since you can keep the foreign currency on a foreign bank/money market account). The forward-spot parity then gives

$$F = e^{m(y-y^*)}S.$$

```
(m,S) = (1,1.2)          #time to expiration, #exchange rate now
(y,y*) = (0.0665,0.05)   #domestic interest rate, foreign interest rate

F = exp(m*(y-y*)) * S

printlnPs("Forward price of foreign currency: ",F)
```

Forward price of foreign currency: 1.220

Bonds 1

This notebook discusses bond price, interest rates, forward rates, yield to maturity and (as extra material) estimates the yield curve.

The [InterestRates.jl](#) and [FinanceModels.jl](#) packages contain many more methods for working with bonds.

Load Packages and Extra Functions

```
using Printf, Roots  
  
include("src/printmat.jl");
```

```
using Plots  
default(size = (480,320),fmt = :png)
```

Interest Rate vs (Zero Coupon) Bond Price

Zero coupon bonds (also called bills or discount bonds) have very simple cash flows: buy the bond now and get the face value (here normalised to 1) at maturity. Clearly, it can be resold at any time.

The bond price (for maturity m years ahead) is a function of the effective interest rate

$$B = (1 + Y)^{-m}$$

and the inverse is $Y = B^{-1/m} - 1$. For instance, $m = 1/12$ is a month and $m = 2$ is two years.

Instead, with a continuously compounded interest rate we have

$$B = e^{-my}$$

$$\text{and } y = -(\ln B)/m.$$

These expressions are coded up as short functions in the next cell.

```

"""
Zero coupon bond/bill price B as a function of Y and m
"""
BillPrice(Y,m) = (1+Y)^(-m)

"""
Effective interest rate Y as a function of B and m
"""
EffRate(B,m) = B^(-1/m) - 1

"""
Zero coupon bond/bill price B as a function of y and m
"""
BillPrice2(y,m) = exp(-m*y)

"""
Continuously compounded interest rate y as a function of B and m
"""
ContCompRate(B,m) = -log(B)/m

```

ContCompRate

Long-maturity bonds are more sensitive to interest rate changes than short-maturity bonds, as illustrated by the plot below.

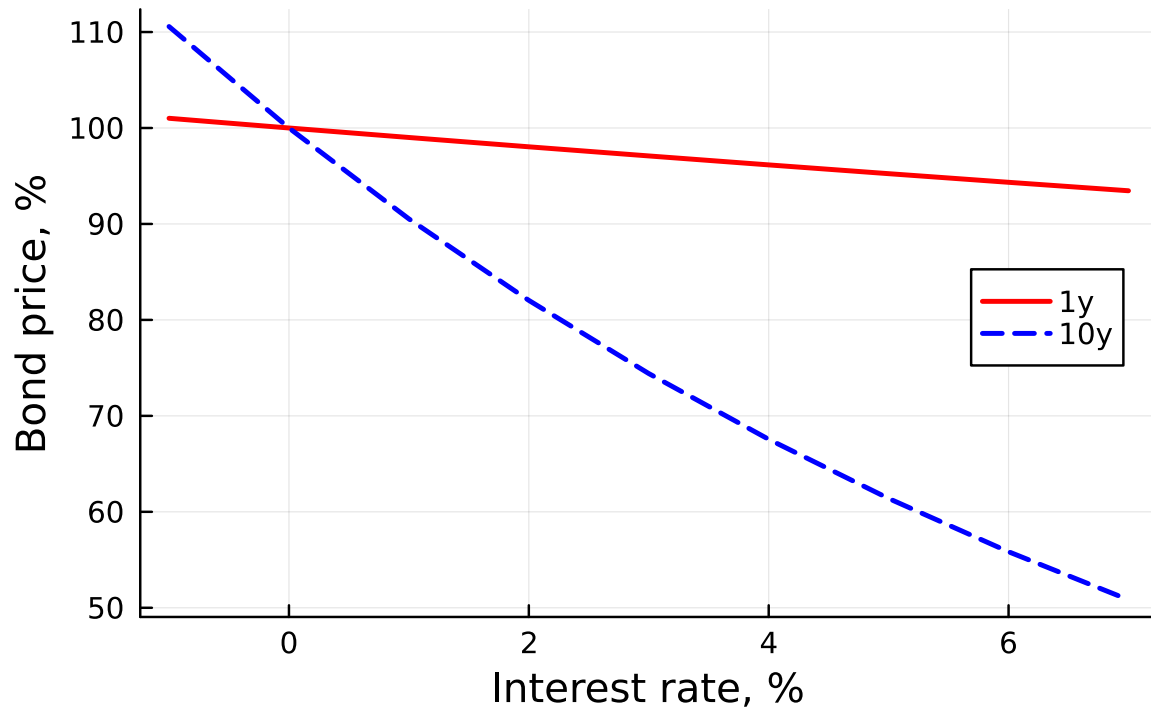
```

Y_range = -0.01:0.01:0.07      #different interest rates
B1       = BillPrice.(Y_range,1) #prices at different interest rates, 1-year zero coupon bond
B10      = BillPrice.(Y_range,10) #10-year bond

p1 = plot( Y_range*100,[B1 B10]*100,
            linecolor = [:red :blue],
            linestyle = [:solid :dash],
            linewidth = 2,
            label = ["1y" "10y"],
            legend = :right,
            title = "Bond prices and rates",
            xlabel = "Interest rate, %",
            ylabel = "Bond price, %" )
display(p1)

```

Bond prices and rates



Effective and continuously compounded interest rates are fairly similar at low interest rates, but start to diverge at high rates.

```
m = 0.5 #maturity, in years
B = 0.95 #bond price

Y = EffRate(B,m) #effective interest rate
y = ContCompRate(B,m) #continuously compounded interest rate

printblue("Interest rates ($m-year bond with a price of $B):")
printmat([Y,y];rowNames= ["Effective","Continuously compounded"])
```

Interest rates (0.5-year bond with a price of 0.95):

Effective	0.108
Continuously compounded	0.103

Bond Returns (Zero Coupon Bonds)

Let B_0 be the bond price in period 0 and B_s the price of the same bond s periods later. The *log* return of holding this bond is

$$r_s = \ln(B_s/B_0).$$

In the computations below, the time to maturity is assumed to be the same in $t = 0$ and $t = s$. This is a reasonable *approximation* if the holding period s are a few days or perhaps weeks, while time to maturity is much longer.

A Remark on the Code

- The code below lets B_0 and B_s be vectors with prices for different maturities.

```

y0 = 0.01                                #(continuously compounded) interest rate before
ys = 0.02                                #interest rate after

B0 = BillPrice2.([y0 y0],[1 10])         #bond prices (1-year and 10 year) before, same interest rates, ve
Bs = BillPrice2.([ys ys],[1 10])         #bond prices (1-year and 10 year) after
rs = log.(Bs./B0)                        #returns

xy      = vcat([y0 y0],B0,[ys ys],Bs,rs)      #to table
rowNames = ["Rate, t=0";"Bond price, t=0";"Rate, t=s";"Bond price, t=s";"Return"]
printblue("Analysis of two bonds: 1y and 10y maturities")
printmat(xy*100;colNames=["1-year";"10-year"],rowNames=rowNames,width=15,prec=2)

printred("Notice that the return is -m*Δy")

```

Analysis of two bonds: 1y and 10y maturities

	1-year	10-year
Rate, t=0	1.00	1.00
Bond price, t=0	99.00	90.48
Rate, t=s	2.00	2.00
Bond price, t=s	98.02	81.87
Return	-1.00	-10.00

Notice that the return is $-m \cdot \Delta y$

Forward Rates

From the forward-spot parity, the forward price of a bond (delivered in $t = m$ and maturing in $t = n$, with time again measured in years) is

$$F = [1 + Y(m)]^m B(n) = B(n)/B(m).$$

A forward interest rate can then be defined as

$$\Gamma(m, n) = F^{-1/(n-m)} - 1.$$

This is the effective interest rate paid (or received) between $t = m$ and $t = n$, but agreed upon already in $t = 0$.

```
"""
    ForwardRate(Ym,m,Yn,n)

Calculate late forward rate for the investment period m to n (where m<n).

# Input
- Ym: interest rate for maturity m
- m: maturity m
- Yn: interest rate for maturity n
- n: maturity n

"""
function ForwardRate(Ym,m,Yn,n)          #forward rate
    if m >= n
        error("m<n is required")
    end
    Bm = (1+Ym)^(-m)
    Bn = (1+Yn)^(-n)
    F  = Bn/Bm
    Γ  = F^(-1/(n-m)) - 1
    return Γ
end
```

ForwardRate

```
(m,n,Ym,Yn) = (0.5,0.75,0.04,0.05)

Γ = ForwardRate(Ym,m,Yn,n)
printlnPs("\nImplied forward rate ($m-year → $n-year): ",Γ)
```

Implied forward rate (0.5-year -> 0.75-year): 0.070

Coupon Bond Prices

Recall that a coupon bond price P is the portfolio value

$$P = \sum_{k=1}^K B(m_k) cf_k, \text{ or}$$

$$P = \sum_{k=1}^K \frac{cf_k}{[1+Y(m_k)]^{m_k}}.$$

where cf_k is the cash flow m_k years ahead and $B(m_k)$ is the price of a zero-coupon bond maturing in the same period. The last cash flow includes also the payment of the face value (if any).

```
B = [0.95,0.9]          #B(1),B(2)
c = 0.06                #coupon rate
cf = [c,1+c]            #cash flows in m=1 and 2

P_components = B.*cf
P = sum(B.*cf)

printblue("\n2-year bond with $c coupon:")
rowNames = ["1-year zero coupon bond price","2-year zero coupon bond price","coupon bond price"]
xx = hcat([B;NaN],[P_components;P])
printmat(xx;rowNames,colNames=["zero-coupon price","zero-coupon price*cf"],width=25)
```

2-year bond with 0.06 coupon:

	zero-coupon price	zero-coupon price*cf
1-year zero coupon bond price	0.950	0.057
2-year zero coupon bond price	0.900	0.954
coupon bond price	NaN	1.011

A Remark on the Code

- The BondPrice() function below can handle both the case when Y is a vector with different values for different maturities and when Y is a scalar (same interest rate for all maturities). It can also handle continuously compounded interest rates by a keyword argument.

```

"""
    BondPrice(Y,cf,m;ContRateQ=false)

Calculate bond price as sum of discounted cash flows.

# Input:
- `Y`: scalar or K vector of interest rates
- `cf`: K vector of cash flows
- `m`: K vector of years to the cash flows
- `ContRateQ`: if true: `Y` is interpreted as a continuously compounded rate
"""
function BondPrice(Y,cf,m;ContRateQ=false)           #cf is a vector of all cash flows at times
    if length(cf) != length(m)
        error("BondPrice: cf and m must have the same lengths")
    end
    if ContRateQ
        cdisc = cf./exp.(m.*Y)
    else
        cdisc = cf./((1.0.+Y).^m)                    #c/(1+y[1])^t1 + c/(1+y[2])^t2 + ...+ c/(1+y[m])^tm
    end
    P = sum(cdisc)                                    #price
    return P
end

```

BondPrice

```

Y = [0.053,0.054]
c = 0.06
P = BondPrice(Y,[c,c+1],[1,2])
printblue("\n2-year bond with $c coupon:")
rowNames = ["1-year spot rate","2-year spot rate","bond price"]
printmat([Y;P];rowNames)

Y = [0.06,0.091]
c = 0.09
P = BondPrice(Y,[c,c+1],[1,2])
printblue("\n2-year bond with $c coupon:")
printmat([Y;P];rowNames)

```

2-year bond with 0.06 coupon:

1-year spot rate	0.053
2-year spot rate	0.054
bond price	1.011

2-year bond with 0.09 coupon:

1-year spot rate	0.060
2-year spot rate	0.091
bond price	1.001

Bootstrapping

With information about coupons $c(m)$ and coupon bond price $P(m)$, we solve for the implied zero coupon bond prices $B(s)$ from a system like (here with just 2 maturities)

$$\begin{bmatrix} P(1) \\ P(2) \end{bmatrix} = \begin{bmatrix} c(1) + 1 & 0 \\ c(2) & c(2) + 1 \end{bmatrix} \begin{bmatrix} B(1) \\ B(2) \end{bmatrix}$$

Notice that each row refers to a specific bond and that each column (in matrix) to different times of cash flows.

A Remark on the Code

$B = \text{cfMat} \backslash P$ solves $P = \text{cfMat} * B$ for B .

```
c      = [0,0.06]
P      = [0.95,1.01]           #coupon bond prices
m      = [1,2]                 #time of coupon payments
cfMat = [1    0    ;           #cash flow matrix
         c[2] 1+c[2]]

println("The cash flow matrix")
printmat(cfMat,colNames=["year 1";"year 2"],rowNames=["Bond 1";"Bond 2"],colUnderlineQ=true)

println("B from solving P = cfMat*B (implied zero-coupon bond prices):")
B = cfMat \ P
printmat(B,rowNames=["1-year";"2-year"])

Y = EffRate.(B,m)             #solve for the implied spot rates
```

```
println("Implied spot interest rates:")
printmat(Y,rowNames=["1-year";"2-year"])
```

The cash flow matrix

	year 1 -----	year 2 -----
Bond 1	1.000	0.000
Bond 2	0.060	1.060

B from solving $P = \text{cfMat} \cdot B$ (implied zero-coupon bond prices):

1-year	0.950
2-year	0.899

Implied spot interest rates:

1-year	0.053
2-year	0.055

Yield to Maturity

The yield to maturity (ytm) is the θ that solves

$$P = \sum_{k=1}^K \frac{cf_k}{(1+\theta)^{m_k}}.$$

We typically have to find θ by a numerical method.

A Remark on the Code

- The [Roots.jl](#) package is used to find the ytm.
- The `find_zero(fn,(lower,upper))` finds a root of the function `fn` in the interval `(lower,upper)`.
- The function is here $\theta \rightarrow \text{BondPrice}(\theta, \text{cf}, m) - P$. This expression creates an anonymous function that takes θ as the only argument, and calculates the difference between the price according to `BondPrice( $\theta$ , cf, m)` and the actual price `P`. The ytm θ makes this zero.

```
c = 0.04           #simple case
Y = 0.03           #all spot rates are 3%
m = [1,2]          #time of cash flows
cf = [c,c+1]       #cash flows

P = BondPrice(Y,cf,m)
```

```

ytm = find_zero(θ→BondPrice(θ,cf,m)-P,(-0.1,0.1))    #solving for ytm

printblue("Price and ytm of 2-year $c coupon bond when all spot rates are $Y:")
printmat([P,ytm],rowNames=["price","ytm"])

```

Price and ytm of 2-year 0.04 coupon bond when all spot rates are 0.03:

```

price    1.019
ytm      0.030

```

```

m  = [1,3]          #cash flow in year 1 and 3
cf = [1,1]          #cash flows
Y  = [0.07,0.10]    #spot interest rates for different maturities

P  = BondPrice(Y,cf,m)
ytm = find_zero(y→BondPrice(y,cf,m)-P,(-0.2,0.2))

printblue("'bond' paying 1 in both t=1 and in t=3")
printmat([Y;ytm];rowNames=["1-year spot rate","3-year spot rate","ytm"])

```

'bond' paying 1 in both t=1 and in t=3

```

1-year spot rate    0.070
3-year spot rate    0.100
ytm                 0.091

```

Estimating Yield Curve with Regression Analysis (extra)

Recall: with a quadratic discount function

$$B(m) = a_0 + a_1 m + a_2 m^2,$$

we can write the coupon bond price

$$P(m_K) = \sum_{k=1}^K B(m_k) c + B(m_K) \text{ as}$$

$$P(m_K) = \sum_{k=1}^K (a_0 + a_1 m_k + a_2 m_k^2) c + (a_0 + a_1 m_K + a_2 m_K^2)$$

Collect terms as

$$P(m_K) = a_0 (Kc + 1) + a_1 (c \sum_{k=1}^K m_k + m_K) + a_2 (c \sum_{k=1}^K m_k^2 + m_K^2)$$

We estimate (a_0, a_1, a_2) by using the terms (within parentheses) as regressors.

A Remark on the Code

$a = x \backslash P$ solves $P = x * a$ if x is square, and computes the least-squares solution when x is rectangular (as in the cell below).

```
c      = [0,0.06]
P      = [0.95,1.01]          #coupon bond prices
m      = [1,2]                #time of coupon payments

n = length(P)
x = zeros(n,3)                #create regressors for quadratic model: 3 columns
for i in 1:n                  #x[i,j] is for bond i, regressor j
    x[i,1] = m[i]*c[i] + 1
    x[i,2] = c[i]*sum(m[1]:m[i]) + m[i]
    x[i,3] = c[i]*sum(abs2,m[1]:m[i]) + m[i]^2    #sum(abs2,x) is the same as sum(x.^2)
end
println("regressors:")
printmat(x,colNames=["term 0";"term 1";"term 2"],rowNames=["Bond 1";"Bond 2"],colUnderlineQ=true)

a = x \ P                     #regress P on x
println("regression coefficients")
printmat(a)

m = 1:5
B = [ones(length(m)) m m.^2]*a    #fitted discount function

printmat(B,colNames=["fitted B"],rowNames=string.(m,"-year"))
printred("Btw. do B[4:5] make sense? If not, what does that teach us?\n")

Y = EffRate.(B[1:2],m[1:2])      #solve for the implied spot rates
printmat(Y,colNames=["fitted Y"],rowNames=string.(m[1:2],"-year"))
```

regressors:

	term 0 -----	term 1 -----	term 2 -----
Bond 1	1.000	1.000	1.000
Bond 2	1.120	2.180	4.300

regression coefficients

```
0.693
0.411
-0.154
```

	fitted B
1-year	0.950
2-year	0.899
3-year	0.540
4-year	-0.126
5-year	-1.101

Btw. do B[4:5] make sense? If not, what does that teach us?

	fitted Y
1-year	0.053
2-year	0.055

Bonds 2

This notebook discusses *duration hedging* as a way to immunize a bond portfolio.

Load Packages and Extra Functions

```
using Printf, Roots  
  
include("src/printmat.jl");
```

```
using Plots  
default(size = (480,320),fmt = :png)
```

A Remark on the Code

The file included below contains the function `BondPrice()` which calculates the present value of a cash flow stream. (It's an extract from the notebook on Bonds 1.)

```
include("src/BondCalculations.jl");
```

Value of a Bond Portfolio after a Sudden Interest Rate Change

The calculations below assume that the *yield curve is flat*, but that it can shift up or down in parallel. This assumption is similar to the classical literature on duration hedging.

The initial values are indicated by the subscript $_0$ and the values after the interest rate change by the subscript $_1$. It is assumed that the change is very sudden, so the time to the different cash flows is (virtually) the same before and after. For instance, the subscripts 0 and 1 could be interpreted as “day 0” and “day 1”.

The next cell sets up the cash flow for a bond portfolio L (“Liability”) that pays the amount 0.2 each year for the next 10 years. The value of L is calculated at an yield to maturity (θ_0) and at a new yield to maturity (θ_1). Notice that with a flat yield curve, the yield to maturity equals the (one and only) interest rate.

Time to cash flow (m) is measured in years.

```
theta_0 = 0.05          #initial interest rate
theta_1 = 0.03          #interest rate after sudden change

cf = ones(10)*0.2      #cash flow of liability, 0.2 in year 1,2,...,10
m = 1:10               #time periods of the cash flows, years

PL_0 = BondPrice(theta_0,cf,m)  #value of bond L at initial interest rate
PL_1 = BondPrice(theta_1,cf,m)  #and after sudden change in interest rate
R = (PL_1 - PL_0)/PL_0         #relative change of the value

printblue("Value of bond portfolio at theta_0=$theta_0 and theta_1=$theta_1:")
xy = [PL_0, PL_1, R]
rowNames = ["PL_0 (at theta_0=$theta_0)"; "PL_1 (at theta_1=$theta_1)"; "delta PL/PL_0 (return)"]
printmat(xy;rowNames,width=15)

printred("Notice that the bond is worth more at the lower interest rate")
```

Value of bond portfolio at $\theta_0=0.05$ and $\theta_1=0.03$:

PL_0 (at $\theta_0=0.05$)	1.544
PL_1 (at $\theta_1=0.03$)	1.706
$\Delta PL/PL_0$ (return)	0.105

Notice that the bond is worth more at the lower interest rate

Durations

Let D^M denote Macaulay’s duration

$$D^M = \sum_{k=1}^K m_k \frac{cf_k}{(1+\theta)^{m_k} P}.$$

It can be interpreted as the weighted average time (years) to cash flow.

The modified duration is

$$D = D^M / (1 + \theta),$$

which can be used in a first-order Taylor approximation to get the (approximate) return

$$\frac{\Delta P}{P} \approx -D \times \Delta \theta.$$

```

"""
    BondDuration(P,cf,m,θ)

Calculate Macaulay's and the modified (bond) duration measures.

P:  scalar, bond price
cf: scalar or K vector of cash flows
m:  K vector of times of cash flows
θ: scalar, yield to maturity
"""
function BondDuration(P,cf,m,θ)
    if length(cf) != length(m)
        error("cf and m must have the same lengths")
    end
    cdisc = cf.*m./((1+θ).^m)      #cf1/(1+y) + 2*cf2/(1+y)^2 + ...
    Dmac   = sum(cdisc)/P          #Macaulays duration
    D       = Dmac/(1+θ)           #modified duration
    return D, Dmac
end

```

BondDuration

```

(DL,Dmac) = BondDuration(PL₀,cf,m,θ₀)
printlnPs("Macaulay's and the modified durations of the bond portfolio L:",Dmac,DL)

Δθ      = θ₁ - θ₀          #change of ytm
R_approx = -DL*Δθ

printblue("\nRelative price change (return): ")
printmat([R,R_approx,];rowNames=["Exact","Approximate (from duration)"])

```

Macaulay's and the modified durations of the bond portfolio L: 5.099 4.856

Relative price change (return):

Exact	0.105
Approximate (from duration)	0.097

Hedging a Liability Stream

Suppose we are short one bond portfolio L (we have a liability) as discussed above, which is worth P_L . To hedge against interest rate changes, we buy v units of bond (portfolio) H . The balance is put on a money market account M to make the initial value of the portfolio zero ($V = 0$)

$$V = vP_H + M - P_L.$$

Over a short time interval, the change (indicated by Δ) in the overall portfolio value is

$$\Delta V = v\Delta P_H - \Delta P_L.$$

In the cells below, we assume that the yield curve is flat and shifts in parallel. This means that the ytm of both L and H change from one common value (θ_0) to another common value (θ_1).

Duration Matching

Duration matching means that we choose a hedge bond with the *same duration* as the liability and invest same amount in each. Clearly, this gives $\frac{\Delta V}{P_L} \approx 0$.

The code below uses a zero coupon bond as bond H , but that is not important. It has the same maturity as Macaulay's duration of the liability.

```
PH0 = BondPrice(theta,1,Dmac)      #value of hedge bond before, here a zero coupon bond
PH1 = BondPrice(theta1,1,Dmac)     #value of hedge bond, after

DH, = BondDuration(PH0,1,Dmac,theta) #modified duration of hedge bond

v = PL0/PH0                        #so v*PH0 = PL0, same amount in L and H

DeltaV = v*(PH1-PH0) - (PL1-PL0)   #value change of total portfolio
R = DeltaV/PL0                     #relative value change

txt = ""Hedge bond: a zero coupon bond with m=$(round(Dmac,digits=2)) and face value of 1.\n
Recall that the interest rates change from theta=$theta to theta1=$theta1.\n
Results of the calculations:""
printblue(txt)

xy = [PL0,PH0,v,v*PH0/PL0,DL,DH,DeltaV]
rowNames = ["PL0","PH0","v","v*PH0/PL0","Duration(liability)","Duration(hedge)","DeltaV"]
printmat(xy;rowNames,prec=2)

printred("The duration matching gives a return, DeltaV/PL0, of $(round(R*100,digits=1))%. Close to zero")
```

Hedge bond: a zero coupon bond with $m=5.1$ and face value of 1.

Recall that the interest rates change from $\theta_0=0.05$ to $\theta_1=0.03$.

Results of the calculations:

PL_0	1.54
PH_0	0.78
v	1.98
$v*PH_0/PL_0$	1.00
Duration(liability)	4.86
Duration(hedge)	4.86
ΔV	-0.00

The duration matching gives a return, $\Delta V/PL_0$, of -0.2%. Close to zero.

Naive Hedging

The “naive” hedging invests the *same amount in the hedge bond* as the value of the liability, that is, $vP_H = P_L$ so $v = P_L/P_H$, but pays no attention to the durations.

The effectiveness of this approach depends on the interest rate sensitivities of L and H . In particular, it can be shown that the relative value change of the overall portfolio is

$$\frac{\Delta V}{P_L} \approx (D_L - D_H) \times \Delta \theta,$$

so it depends on the difference between the durations (of the liability and hedge bond).

In particular, if $D_L > D_H$, and $\Delta \theta < 0$ (as in the example below), then we will lose money. More importantly, with $D_L \neq D_H$, we are taking a risk (of losses/gains), that is, we are not hedged.

In contrast, with $D_L = D_H$, then we are effectively doing duration matching (see above). Clearly, there are also scenarios

```
mH = 3                                #a mH-year zero coupon bond is used as hedge bond

PH0 = BondPrice(theta0,1,mH)
PH1 = BondPrice(theta1,1,mH)
DH, = BondDuration(PH0,1,mH,theta0)    #modified duration of hedge bond

v = PL0/PH0                            #so v*PH0 = PL0

DeltaV = v*(PH1-PH0) - (PL1-PL0)
R = DeltaV/PL0                          #relative value change
```

```

printblue("Hedge bond: zero coupon bond with m=$mH and face value of 1\n")
xy = [PLθ,PHθ,v,v*PHθ/PLθ,DL,DH,ΔV]
rowNames = ["PLθ","PHθ","v","v*PHθ/PLθ","Dur(liability)","Dur(hedge)","ΔV"]
printmat(xy;rowNames,prec=2)

printred("The naive hedge here gives a return, ΔV/PLθ, of $(round(R*100,digits=1))%, which is a bad

```

Hedge bond: zero coupon bond with m=3 and face value of 1

PL _θ	1.54
PH _θ	0.86
v	1.79
v*PH _θ /PL _θ	1.00
Dur(liability)	4.86
Dur(hedge)	2.86
ΔV	-0.07

The naive hedge here gives a return, ΔV/PL_θ, of -4.5%, which is a bad hedge

Illustrating the Problem with the Naive Hedging

by plotting the value of the liability (P_L) and of the hedge bond position (vP_H) at different interest rates.

A Remark on the Code

The next cell calculates PL and PH by a list comprehension. To instead do it with a loop use something like

```

PL = fill(NaN,length(θ_range))
for i in 1:length(θ_range)
    PL[i] = BondPrice(θ_range[i],cf,m)
end

```

```

θ_range = 0:0.01:0.1           #possible ytm values

PL = [BondPrice(θ,cf,m) for θ in θ_range] #list comprehension,
PH = [BondPrice(θ,1,mH) for θ in θ_range] #an alternative to a loop

```

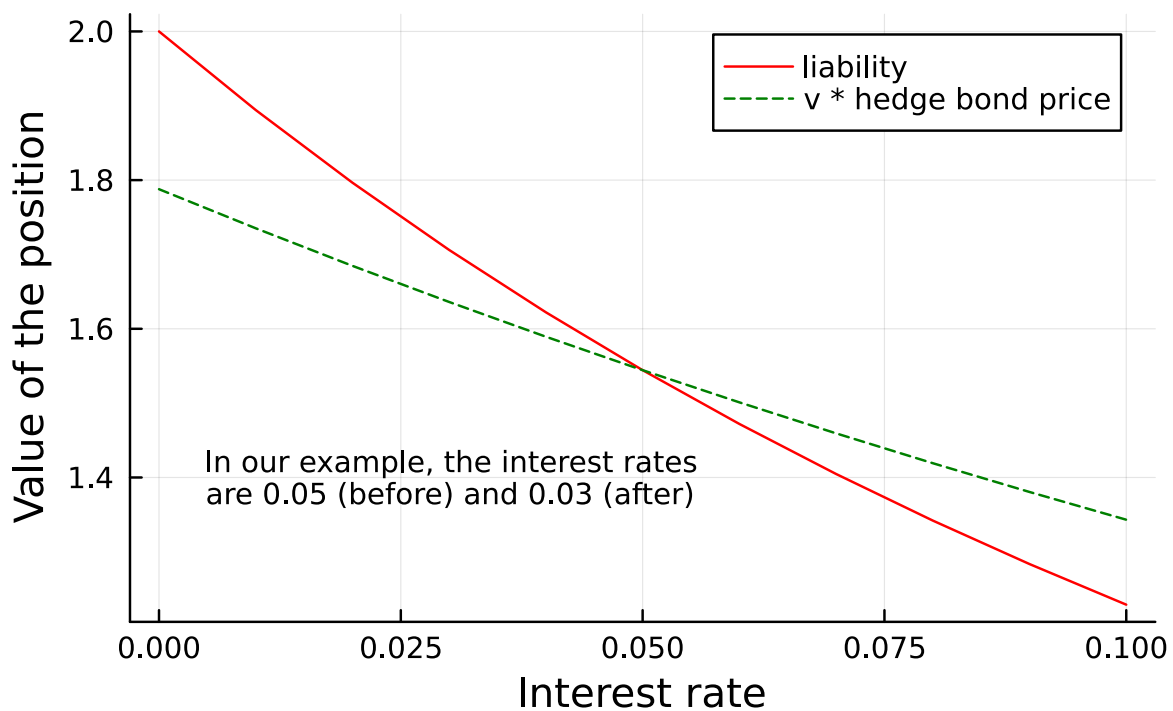
```

txt = "In our example, the interest rates \nare $0_0 (before) and $0_1 (after)"

p1 = plot( 0_range,[PL v*PH],
          linecolor = [:red :green],
          linestyle = [:solid :dash],
          label = ["liability" "v * hedge bond price"],
          title = "Naive hedging",
          xlabel = "Interest rate",
          ylabel = "Value of the position",
          annotation = (0.005,1.4,text(txt,8,:left)))
display(p1)

```

Naive hedging



Duration Hedging

With $D_L \neq D_H$, we could *adjust the hedge ratio* v to compensate for the difference in interest rate sensitivity (duration). In particular, we could set

$$v = \frac{D_L}{D_H} \times \frac{P_L}{P_H}.$$

The balance ($P_L - vP_H$) is kept on a money market account (M).

It can be shown that this gives an (approximate) hedge.

```
PH0 = BondPrice(theta,1,mH)
PH1 = BondPrice(theta1,1,mH)
v = DL/DH * PL0/PH0
M = PL0 - v*PH0 #on money market account

DeltaV = v*(PH1-PH0) - (PL1-PL0)
R = DeltaV/PL0 #relative value change

printblue("Hedge bond: zero coupon bond with m=$mH and face value of 1\n")
xy = [PL0,PH0,v,v*PH0/PL0,DL,DH,M,DeltaV]
rowNames = ["PL0","PH0","v","v*PH0/PL0","Dur(liability)","Dur(hedge)","M","DeltaV"]
printmat(xy,rowNames,prec=2)

printred("The duration hedging gives a return, DeltaV/PL0, of $(round(R*100,digits=1))%. Close to zero.
```

Hedge bond: zero coupon bond with m=3 and face value of 1

PL ₀	1.54
PH ₀	0.86
v	3.04
v*PH ₀ /PL ₀	1.70
Dur(liability)	4.86
Dur(hedge)	2.86
M	-1.08
ΔV	-0.01

The duration hedging gives a return, $\Delta V/PL_0$, of -0.4%. Close to zero.

Convexity (extra)

A second-order Taylor approximation gives that

$$\frac{\Delta P}{P} \approx -D \times \Delta \theta + \frac{1}{2} C \times (\Delta \theta)^2,$$

where $C = \frac{1}{P} \frac{d^2 P}{d\theta^2}$.

The function below calculates C .


```

"""
    BondConvexity(P,cf,m,ym)

Calculate the convexity term 'C'. The effect on the bond is 0.5C*(Δym)^2
"""
function BondConvexity(P,cf,m,ym)
    cdisc = cf.*m.*(m.+1)./((1+ym).^(m.+2))
    C      = sum(cdisc)/P
    return C
end

```

BondConvexity

```

C = BondConvexity(PL0,cf,m,θ0)

printblue("Convexity of the liability")
xy = [C;Δθ;0.5*C*Δθ^2]
printmat(xy,rowNames=["C";"Δθ";"0.5*C*(Δθ)^2"])

printred("Compare this magnitude to ΔPL/PL: ",round((PL1-PL0)/PL0,digits=3),". It seems convexity i

```

Convexity of the liability	
C	35.602
Δθ	-0.020
0.5*C*(Δθ)^2	0.007

Compare this magnitude to ΔPL/PL: 0.105. It seems convexity is not important in this case.

Bonds 3

This notebook presents a yield curve model (a simplified Vasicek model) and then uses it to improve the duration hedging approach.

Load Packages and Extra Functions

```
using Printf

include("src/printmat.jl")
include("src/BondCalculations.jl");
```

```
using Plots, LaTeXStrings
default(size = (480,320),fmt = :png)
```

Predictions from an AR(1)

Consider an AR(1) with mean μ

$$r_{t+1} - \mu = \rho (r_t - \mu) + \varepsilon_{t+1}$$

The forecast (based on information in t) for $t + s$ is

$$E_t r_{t+s} = (1 - \rho^s) \mu + \rho^s r_t.$$

In the the cells below, we code a function for these forecasts and then plot the results (for many forecasting horizons and also for several different starting values).

```

"""
    AR1Prediction(r0,ρ,μ,s)

Calculate forecast from AR(1)

# Input
r0,ρ,μ,s: scalars
"""
AR1Prediction(r0,ρ,μ,s) = (1-ρ^s)*μ + ρ^s*r0;

```

```

(μ,ρ) = (0.05,0.975)

printlnPs("Prediction for t+50, assuming current r=0.07: ",AR1Prediction(0.07,ρ,μ,50))

```

Prediction for t+50, assuming current r=0.07: 0.056

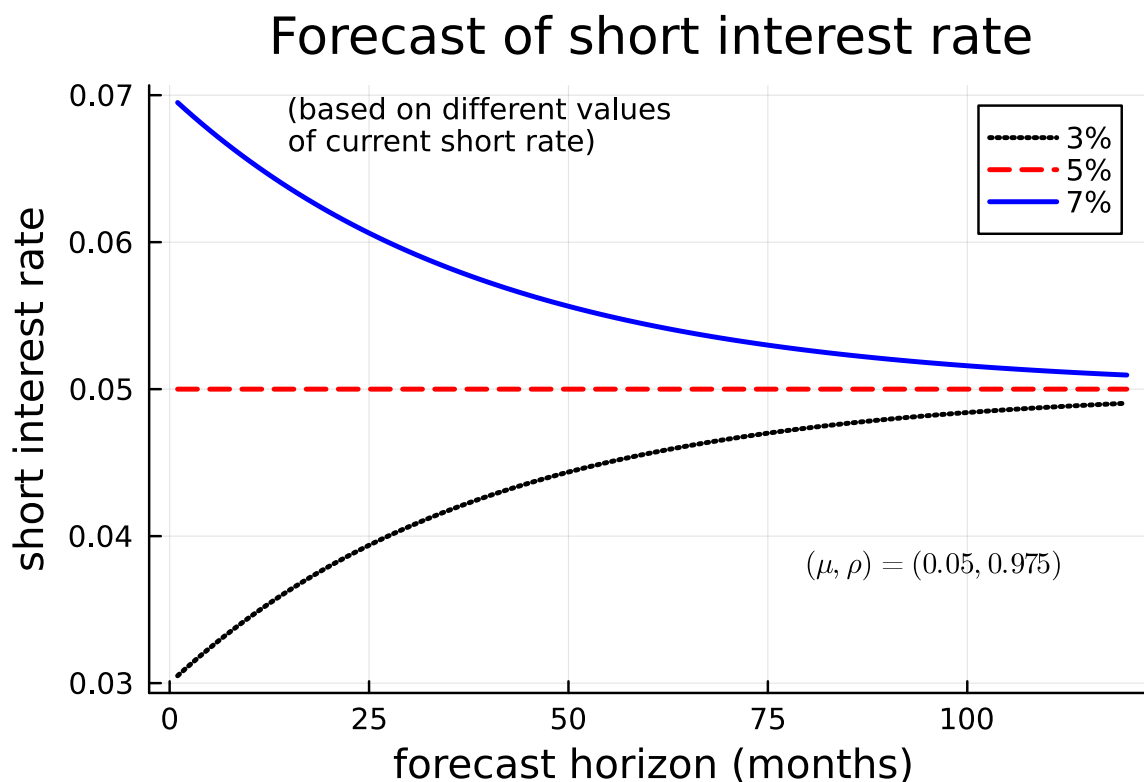
```

sMax      = 120                                #make forecasts for many horizons and initial values
s_range   = 1:sMax                             #different horizons, months
r0        = [0.03,0.05,0.07]

xPred = fill(NaN,sMax,length(r0))              #xPred[maturity,r0 value]
for s in s_range, j in 1:length(r0)
    xPred[s,j] = AR1Prediction(r0[j],ρ,μ,s)
end

txt = text(L"\mu,\rho) = (%μ,%ρ)",8,:left)    #LaTeX string to plot, notice %$
p1 = plot( s_range,xPred,
    linecolor = [:black :red :blue],
    linewidth = 2,
    linestyle = [:dot :dash :solid],
    label = ["3%" "5%" "7%"],
    title = "Forecast of short interest rate",
    xlabel = "forecast horizon (months)",
    ylabel = "short interest rate",
    annotation = [(80,0.038,txt),
        (15,0.068,text("(based on different values\nof current short rate)",8,:left))]
display(p1)

```



The Vasicek Model

The simplified (because of no risk premia) Vasicek model implies that

$$y_t(n) = a(n) + b(n)r_t, \text{ where}$$

$$b(n) = (1 - \rho^n) / [(1 - \rho)n],$$

$$a(n) = \mu [1 - b(n)].$$

In the example, $y_t(36)$ which corresponds to $y[36]$ in the code, is the (annualized) continuously compounded interest rate for a bond maturing in 36 months (3 years).

In the cells below, we code a function for this model and then plots the results for many different maturities (horizons) and several initial values of the short interest rate.

A Remark on the Code

The code (used below) `b = rho == 1.0 ? 1.0 : (1-rho^n)/((1-rho)*n)` is the same as

```

if p == 1.0
    b = 1.0
else
    b = (1-p^n)/((1-p)*n)
end

```

```

"""
    Vasicek(r,ρ,μ,n)

Vasicek model: calculate interest rate and (a,b) coeffs

# Input
r,ρ,μ,n: scalars

n is the maturity, but this could be measured in any unit (months, years, etc),
but changing the units require recalibrating ρ
"""
function Vasicek(r,ρ,μ,n)
    b = ρ == 1.0 ? 1.0 : (1-ρ^n)/((1-ρ)*n)    #if-else-end
    a = μ*(1-b)
    y = a + b*r
    return y,a,b
end

```

Vasicek

```

(y,a,b) = (fill(NaN,sMax,length(r₀)),fill(NaN,sMax),fill(NaN,sMax))
for n in s_range, j in 1:length(r₀)
    #local a, b                                #local/global is needed in script
    (y[n,j],a[n],b[n]) = Vasicek(r₀[j],ρ,μ,n)    #(a,b) are actually the same across r₀ values
end

printblue("a and b for the first few horizons (months):")
printmat([a[1:4]*10 b[1:4]];colNames=["a*10","b"],rowNames=string.(s_range[1:4]),cell00="horizon")

```

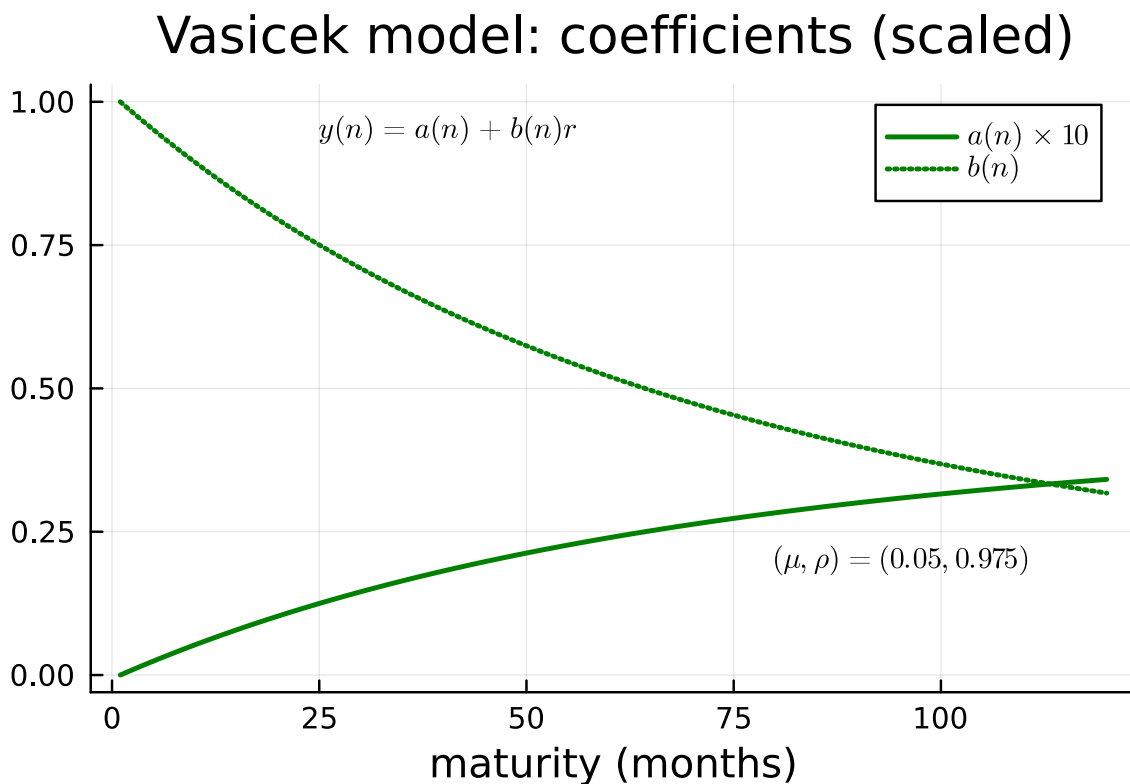
a and b for the first few horizons (months):

horizon	a*10	b
1	0.000	1.000
2	0.006	0.988
3	0.012	0.975
4	0.018	0.963

```

p1 = plot( s_range,hcat(a*10,b),
           linecolor = [:green :green],
           linestyle = [:solid :dot],
           linewidth = 2,
           label = [L"a(n) \times 10" L"b(n)"],
           title = "Vasicek model: coefficients (scaled)",
           xlabel = "maturity (months)",
           annotation = [(80,0.2,txt),
                        (25,0.95,text(L"y(n) = a(n) + b(n)r",8,:left))] )
display(p1)

```



```

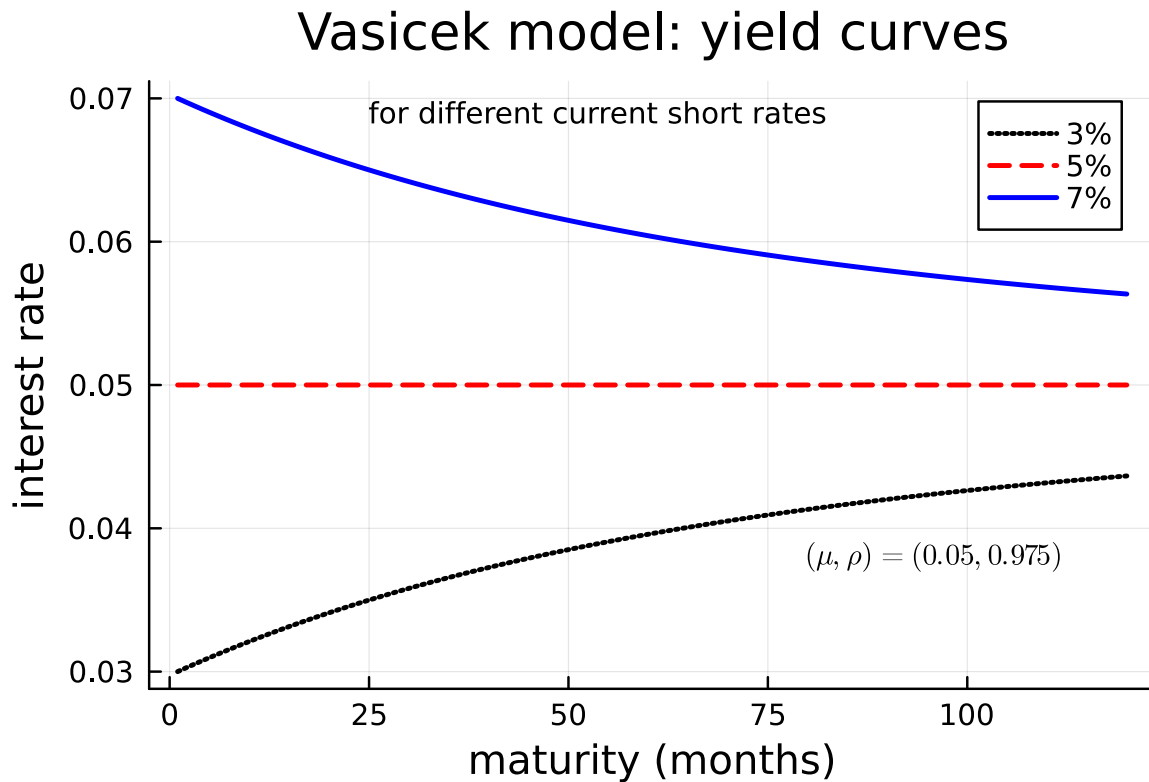
p1 = plot( s_range,y,
           linecolor = [:black :red :blue],
           linewidth = 2,
           linestyle = [:dot :dash :solid],
           label = ["3%" "5%" "7%"],
           title = "Vasicek model: yield curves",
           xlabel = "maturity (months)",
           ylabel = "interest rate",

```

```

annotation = [(80,0.038,txt),
              (25,0.069,text("for different current short rates",8,:left))] )
display(p1)

```



Hedging Using the Vasicek Model

The change of the (value of the) overall portfolio is

$$\Delta V = v \Delta P_H - \Delta P_L$$

The code below calculates the v value that makes $\Delta V \approx 0$, that is, $v = \Delta P_L / \Delta P_H$.

Calculate Yield Curves

This is similar to the previous figure on the Vasicek yield curves, but here for the horizons that we need in order to (later) calculate the prices of the liability and hedge bond.

Notice that (in our application) the periods in the Vasicek model are months, that is, corresponding to the maturity of the short rate. This means, for instance, that the 3-year spot rate is element 36 of the y vector calculated before. To keep the annual periods for the bonds in the hedging exercise, we define nL and nH to be the months of cash flows of the bonds.

```
(cfL,mL) = (fill(0.2,10),1:10)    #cash flows, time of cash flow, liability, 0.2 in year 1,2...,10
(cfH,mH) = (1,3);                  #hedge bond, 3-year zero-coupon bond, just one cash flow

nL = indexin(mL*12,s_range)        #indices in s_range (months) that fit with mL (years), used for t
nH = indexin(mH*12,s_range);

y0 = y[:,2];                        #use the yield curve (calculated above) for  $r_0=5$ 
```

Hedge Ratio from Spot Durations

We now apply a spot duration approach instead.

The `BondDurationSpot()` function calculates spot rate durations, D_k for $k = 1, \dots, K$, that is, derivatives of the bond price wrt. the (vector of) spot rates. This can be used to calculate (approximate) bond price values as

$$\Delta P = -P[\sum_{k=1}^K D_k b(m_k)]\Delta r,$$

where $b(m_k)$ is the coefficient in the Vasicek model for horizon m_k . As before, remember that the output from the Vasicek model is indexed by months (not years). Apply the equation to the liability and to the hedge bond (which have different D_k values).

As before, the hedge ratio is calculated as

$$v = \Delta P_L / \Delta P_H,$$

so the Δr terms cancel.

```
"""
    BondDurationSpot(Y,cf,m;ContRateQ=false)

Calculate (modified) spot rate durations.

# Input
- `Y::Vector`:      spot rates
- `cf::Vector`:     cash flows
- `m::Vector`:      time to cash flows
- `P::Number`:      bond price
- `ContRateQ::Bool`: if `true`: continuously compounded rates
```



```

"""
function BondDurationSpot(Y,cf,m;ContRateQ=false)
    P = BondPrice(Y,cf,m;ContRateQ)      #implied bond price
    if ContRateQ
        D = m.*cf.*exp.(-m.*Y)
    else
        D = m.*cf./(1.0.+Y).^(m.+1)
    end
    D = D/P
    return D, P
end

```

BondDurationSpot

Information about the Two Bonds

In the example below, the liability pays 0.2 every 12 months for 10 years. The hedge bond is a 3-year zero coupon bond.

```

(DurSpotL,PL) = BondDurationSpot(y0[nL],cfL,mL;ContRateQ=true) #spot duration, price
ΔPL = -PL*sum(b[nL].*DurSpotL)
(DurSpotH,PH) = BondDurationSpot(y0[nH],cfH,mH;ContRateQ=true)
ΔPH = -PH*sum(b[nH].*DurSpotH) #combing b from Vasicek with spot durations
v = ΔPL/ΔPH

printblue("Hedge ratio from spot durations combined with the Vasicek model")
xy = [v;v*PH/PL]
printmat(xy,rowNames=["v","v*PH/PL"])

printred("Notice: ρ is important for the hedge ratio v.
Try also ρ=1 (change in one of the first cells in the notebook)
to see how it affects the hedge ratio v.")

```

```

Hedge ratio from spot durations combined with the Vasicek model
v                2.093
v*PH/PL          1.174

```

Notice: ρ is important for the hedge ratio v .
Try also $\rho=1$ (change in one of the first cells in the notebook)
to see how it affects the hedge ratio v .

Options 1

This notebook shows profit functions and pricing bounds for options.

Load Packages and Extra Functions

```
using Printf  
  
include("src/printmat.jl");
```

```
using Plots, LaTeXStrings  
default(size = (480,320),fmt = :png)
```

Profits of Options

Let K be the strike price, S_m the price of the underlying at expiration (m years ahead) of the option contract and y the continuously compounded interest rate.

The call and put profits (at expiration) are

$$\text{call profit}_m = \max(0, S_m - K) - e^{my}C$$

$$\text{put profit}_m = \max(0, K - S_m) - e^{my}P,$$

where C and P are the call and put option prices (paid today). In both cases the first term ($\max()$) represents the payoff at expiration, while the second term ($e^{my}C$ or $e^{my}P$) subtracts the capitalised value of the option price (premium) paid at inception of the contract.

The profit of a straddle is the sum of those of a call and a put.

A Remark on the Code

- S_m _range is a vector (or range) of S_m values. The idea is to show the payoff (or profit) at different possible outcomes of the final price of the underlying.
- S_m _range > K creates a vector of false/true. Notice that the dot (.) is needed to compare each element in S_m _range to the number K.
- ifelse.(S_m _range.>K,"yes","no") creates a vector of “yes” or “no” depending on whether S_m _range.>K or not. This is one of several possible ways of writing an if statement
- To get LaTeX in the graphs use L"some text, $\cos x$ " (Notice the L which indicates that this is a string with some LaTeX.)

```
S_m_range = [4.5,5.5]    #possible values of underlying at expiration
K = 5                  #strike price
C = 0.4                #call price (just a number that I made up)
P = 0.4                #put price
(y,m) = (0,1)          #zero interest to keep it simple, 1 year to expiration
```

```
CallPayoff = max.(0,S_m_range.-K)          #payoff at expiration
CallProfit = CallPayoff .- exp(m*y)*C      #profit at expiration
```

```
ExerciseIt = ifelse.(S_m_range.>K,"yes","no") # "yes"/"no" for exercise
```

```
printblue("Payoff and profit of a call option with strike price $K, price (premium) of $C and interest rate $m")
printmat(S_m_range,ExerciseIt,CallPayoff,CallProfit;colNames=["S_m","Exercise","Payoff","Profit"])
```

Payoff and profit of a call option with strike price 5, price (premium) of 0.4 and interest rate 0:

S_m	Exercise	Payoff	Profit
4.500	no	0.000	-0.400
5.500	yes	0.500	0.100

```
S_m_range = 0:0.1:10    #more possible outcomes, for plotting
```

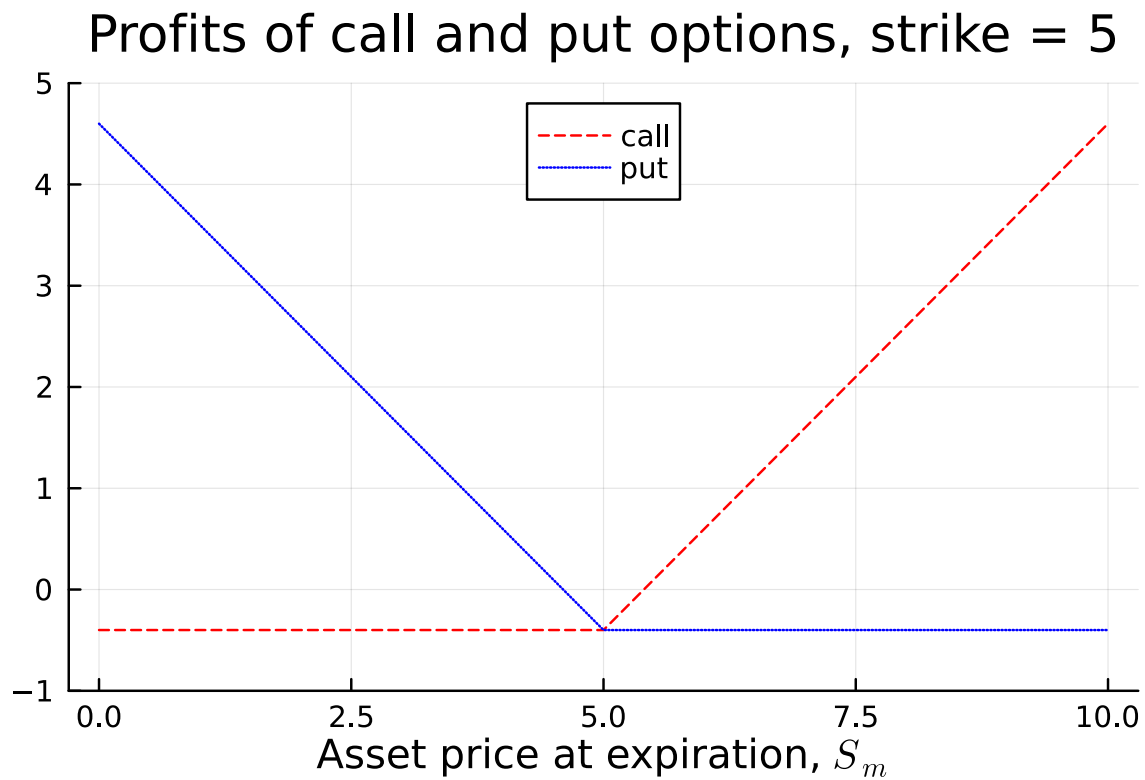
```
CallProfit = max.(0,S_m_range.-K) .- exp(m*y)*C
PutProfit = max.(0,K.-S_m_range) .- exp(m*y)*P
```

```
p1 = plot( S_m_range,[CallProfit PutProfit],
           linecolor = [:red :blue],
           linestyle = [:dash :dot],
           linewidth = 1,
           label = ["call" "put"],
           ylim = (-1,5),
```

```

legend = :top,
title = "Profits of call and put options, strike = $K",
xlabel = L"Asset price at expiration,  $S_m$ " )
display(p1)

```



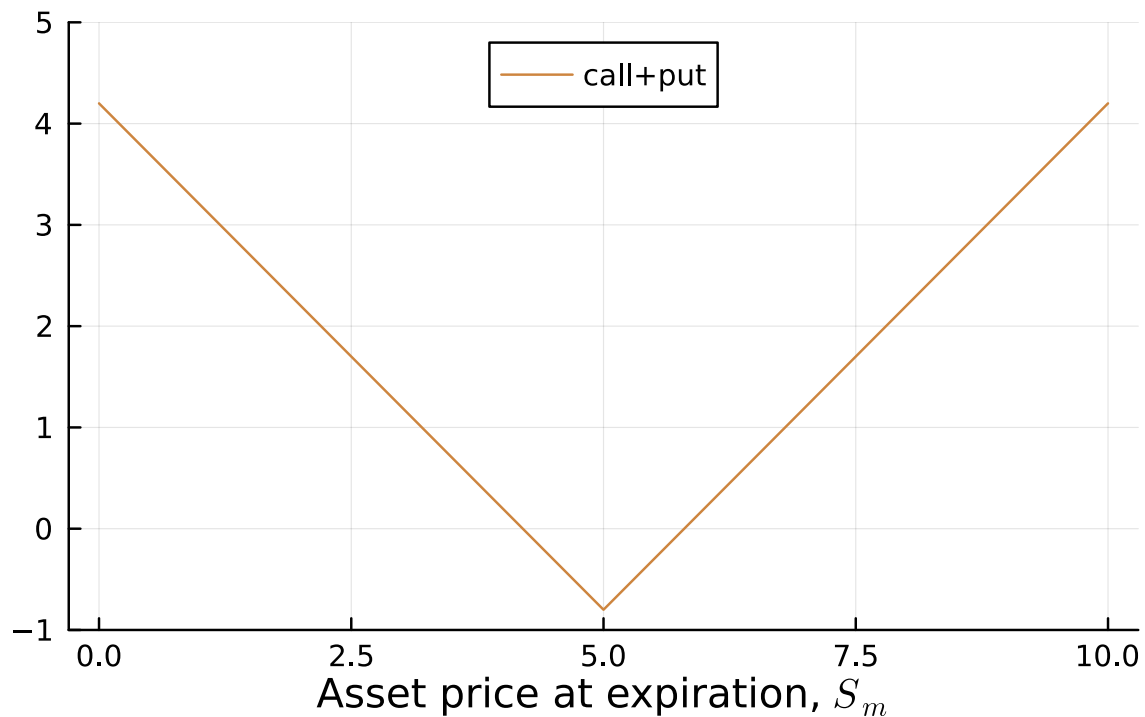
```

StraddleProfit = CallProfit + PutProfit    #a straddle: 1 call and 1 put

p1 = plot( S_m_range, StraddleProfit,
           linecolor = :peru,
           linewidth = 1,
           label = "call+put",
           ylim = (-1,5),
           legend = :top,
           title = "Profit of a straddle, strike = $K",
           xlabel = L"Asset price at expiration,  $S_m$ " )
display(p1)

```

Profit of a straddle, strike = 5



Put-Call Parity for European Options

A no-arbitrage condition says that

$$C - P = e^{-my}(F - K)$$

must hold, where F is the forward price (on a forward contract with the same time to expiration, m , as the option). This is the *put-call parity*.

When the underlying asset has no dividends (until expiration of the option), then the forward-spot parity says that $F = e^{my}S$, which can be used in the put-call parity to substitute for F .

```
(S,K,m,y) = (42,38,0.5,0.05)    #current price of underlying etc
C = 5.5                          #assume this is the price of a call option(K)
F = exp(m*y)*S                  #forward-spot parity
P = C - exp(-m*y)*(F-K)         #put price implied by the parity
```

```
printblue("Put-Call parity when (C,S,y,m)=( $C, S, y, m$ ):\n")

printmat([C,exp(-m*y),F-K,P],rowNames=["C","exp(-m*y)","F-K","P (implied)"])
```

Put-Call parity when $(C,S,y,m)=(5.5,42,0.05,0.5)$:

C	5.500
$\exp(-m*y)$	0.975
F-K	5.063
P (implied)	0.562

Pricing Bounds

The pricing bounds for (American and European) call options are

$$\max[0, e^{-my}(F - K)] \leq C \leq e^{-my}F$$

We plot these bounds as functions of the strike price K .

```
(S,K,m,y) = (42,38,0.5,0.05)      #current price of underlying etc

F = exp(m*y)*S

C_Upper = exp(-m*y)*F
C_Lower = max.(0,exp(-m*y)*(F-K))  #pricing bounds for a (single) strike price

printlnPs("Pricing bounds for European call option with strike $K: ",C_Lower,C_Upper)
```

Pricing bounds for European call option with strike 38: 4.938 42.000

```
K_range = 30:0.5:50                #pricing bounds for many strike prices
n = length(K_range)

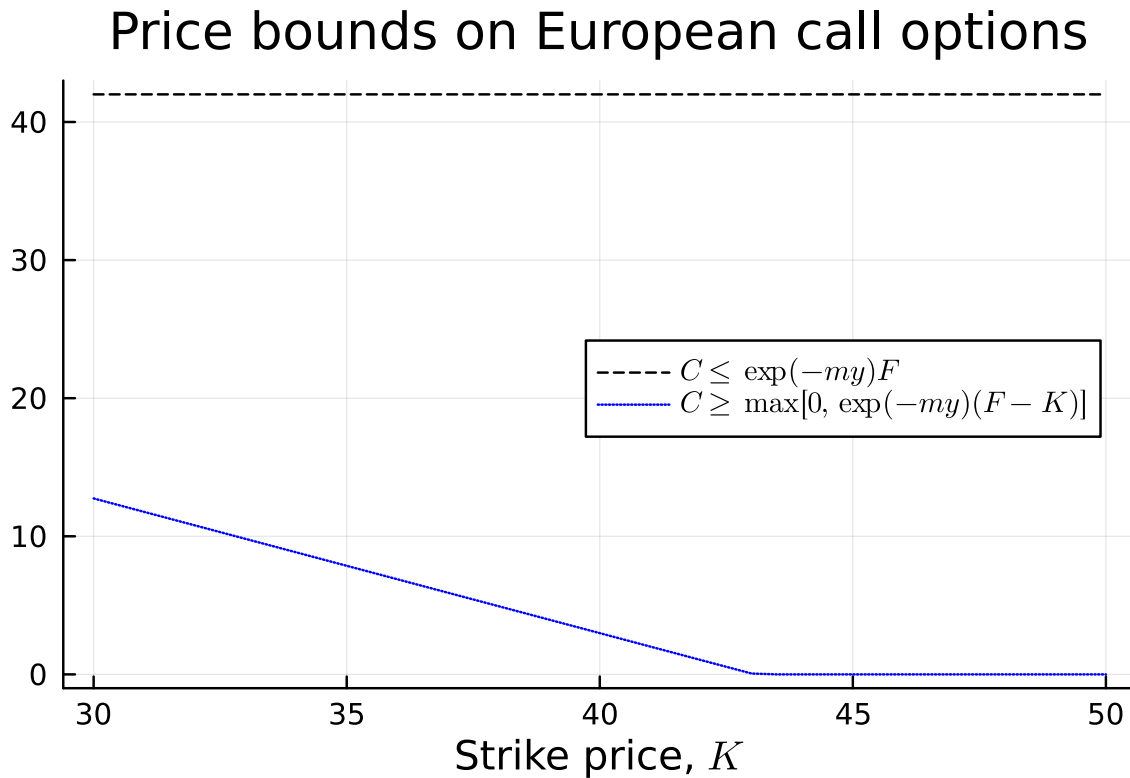
C_Upper = exp(-m*y)*F
C_Lower = max.(0,exp(-m*y)*(F.-K_range));

p1 = plot( K_range,[C_Upper*ones(n) C_Lower],
           linecolor = [:black :blue],
           linewidth = 1,
           linestyle = [:dash :dot],
```

```

label = [L"C \leq \exp(-my)F " L"C \geq \max[0,\exp(-my)(F-K)]"],
ylim = (-1,S+1),
legend = :right,
title = "Price bounds on European call options",
xlabel = L"Strike price, $K$" )
display(p1)

```



The pricing bounds for (European) put options are

$$e^{-my}(K - F) \leq P_E \leq e^{-my}K$$

```

P_Upper = exp(-m*y)*K_range
P_Lower = max.(0,exp(-m*y)*(K_range.-F))

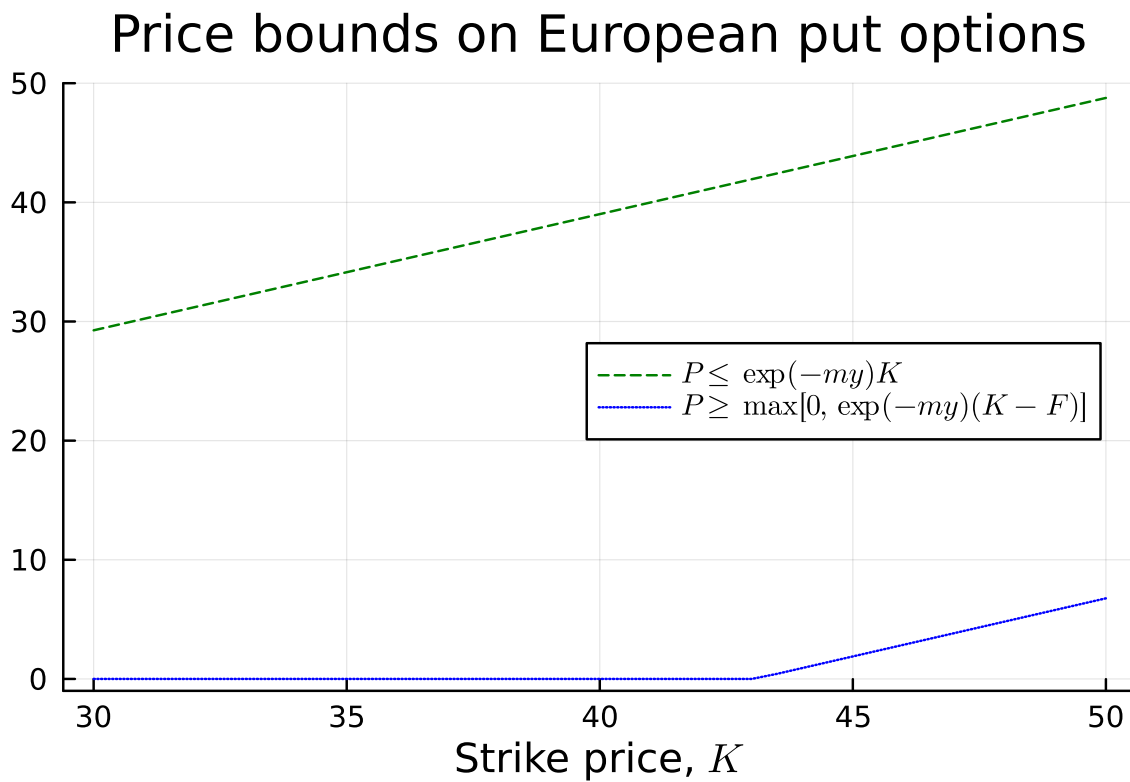
p1 = plot( K_range,[P_Upper P_Lower],
           linecolor = [:green :blue],
           linewidth = 1,
           linestyle = [:dash :dot],
           label = [L" P \leq \exp(-my)K " L"P \geq \max[0,\exp(-my)(K-F)]"],
           ylim = (-1,50),

```

```

legend = :right,
title = "Price bounds on European put options",
xlabel = L"Strike price, $$$" )
display(p1)

```



Options 2: The Binomial Option Pricing Model

This notebook implements the binomial option pricing model for European and American style options. The approach is to set up, fill and store all important arrays (price of underlying asset, exercise decision, option value...in all periods and states) so we can analyze them. Code aimed at performance (rather than teaching) would avoid that.

Load Packages and Extra Functions

The [OffsetArrays.jl](#) package allows flexible indexing of an array. In particular, it allows us to refer to the first element of a vector as element 0. This is useful in order to stick close to lecture notes on binomial trees where period 0 is the starting point and we take n time steps.

```
using Printf, OffsetArrays

include("src/printmat.jl");
```

```
using Plots
default(size = (480,320),fmt = :png)
```

The Binomial Model for *One* Time Step

In a binomial model the price of the underlying asset can change from S today to either S_u or S_d in the next period (which is h years from now).

Let f_u and f_d be the values of the derivative in the up- and down states (next period). Then, the value of the derivative today (f) is

$$f = e^{-yh} [pf_u + (1-p)f_d] \quad \text{where } p = \frac{e^{yh}-d}{u-d}$$

Notice that p denotes the “risk-neutral probability” of an up move. (Don’t mix it with notation for a put price.)

As an example, for a call option that expires next period, the payoffs in the two states are

$$f_u = \max(Su - K, 0)$$

$$f_d = \max(Sd - K, 0)$$

For these equations to make sense, we need $u > e^{yh} > d$, which needs to be checked in any numerical implementation.

```
(S,K,y) = (10,10,0)      #underlying price today, strike price, interest rate
(u,d,h) = (1.1,0.95,1/12) #up move, down move, length of time period

fu = max(S*u-K,0)        #value of call option in up node
fd = max(S*d-K,0)        #in down node

p = (exp(y*h)-d)/(u-d)    #risk-neutral probability of "up" move
CallPrice = exp(-y*h)*(p*fu+(1-p)*fd) #call option price today

printblue("Pricing of a call option with strike $K, one time step:\n")
printmat([fu,fd,p,CallPrice];rowNames=["Payoff 'up'", "Payoff 'down'", "p", "Call price now"])
```

Pricing of a call option with strike 10, one time step:

Payoff 'up'	1.000
Payoff 'down'	0.000
p	0.333
Call price now	0.333

A CRR Tree for *Many* (Short) Time Steps

We now build a tree with n steps of (time), each of length h , to reach expiration m (so $nh = m$).

The CRR approach to construct (u, d) is

$$u = e^{\sigma\sqrt{h}} \text{ and } d = 1/u,$$

where σ is the annualized standard deviation of the return on the underlying asset. Notice that p depends on the choice of (u, d) and also on e^{yh}

$$p = \frac{e^{yh} - d}{u - d}.$$

With these choices, $u > e^{yh} > d$ holds if h is sufficiently small.

```

"""
CRRparams( $\sigma$ ,m,n,y)

    BOPM parameters according to CRR
"""
function CRRparams( $\sigma$ ,m,y,n)
    h = m/n                #time step size (in years)
    u = exp( $\sigma$ *sqrt(h))    #up move
    d = 1/u                 #down move
    p = (exp(y*h) - d)/(u-d) #rn prob of up move
    return h,u,d,p
end

```

CRRparams

```

(m,y, $\sigma$ ,n) = (0.5,0.05,0.2,26)
(h,u,d,p) = CRRparams( $\sigma$ ,m,y,n)

printblue("CRR parameters in tree when m=$m, y=$y,  $\sigma$ =$ $\sigma$  and n=$n:\n")
xx = [h,u,d,p,exp(y*h)]
printmat(xx;rowNames=["h","u","d","p","exp(yh)"])

printblue("Checking if u > exp(y*h) > d: ", u > exp(y*h) > d)

```

CRR parameters in tree when m=0.5, y=0.05, σ =0.2 and n=26:

```

h          0.019
u          1.028
d          0.973
p          0.510
exp(yh)    1.001

```

Checking if u > exp(y*h) > d: true

Build a Tree for the Underlying Asset

The next few cells explain how we build a tree with n time steps for the underlying asset.

Creating a Vector of Vectors

of different lengths.

A Remark on the Code

The next cell illustrates how we can create a vector of vectors (of different lengths). We use the `OffsetArrays.jl` package to set up the outer vector so that the first element has index 0. All the inner vectors are traditional, that is, the first index is 1.

In this example, `x[0]` is a vector `[0]`, while `x[1]` is a vector `[0,0]`, etc.

Using a vector of vector is more straightforward than filling half a matrix, and wastes less memory space.

```
x = [zeros(i+1) for i = 0:2]      #a vector or vectors (of different lengths)
x = OffsetArray(x,0:2)           #convert so the indices of x are 0:2

printblue("Illustrating a vector of vectors (of different lengths):\n")
for i in 0:2
    printblue("x[$i]:")
    printmat(x[i])
end
```

Illustrating a vector of vectors (of different lengths):

```
x[0]:
 0.000
```

```
x[1]:
 0.000
 0.000
```

```
x[2]:
 0.000
 0.000
 0.000
```

Steps (Up and Down)

We create a tree by starting at the current spot price S . The subsequent nodes are then created by multiplying by u or d . That is, the tree (vector of vectors) is built from the first (0) time step to the last (n) time step.

step 0 : (S) as `STree[0]`

step 1 : (Su, Sd) as `STree[1]`

step 2 : (Suu, Sud, Sdd) (since $Sud = Sdu$) as `STree[2]`

...

A Remark on the Code

Each time step is a vector and the vector of those vectors is the entire tree. For instance, for step 2, the code in the next cell creates the vector (Suu, Sud, Sdd) by first multiplying the step 1 vector by the up factor (to get $u(Su, Sd)$) and then attaching (as the last element) the last element of the step 1 vector times the down factor (to get dSd).

```
"""
    BuildSTree(S,n,u,d)

Build binomial tree, starting at 'S' and having 'n' steps with up move 'u' and down move 'd'

# Output
- 'STree:: Vector of vectors': each (sub-)vector is for a time step. 'STree[0] = [S]' and 'STree[n]'

"""
function BuildSTree(S,n,u,d)
    STree = [fill(NaN,i) for i = 1:n+1] #vector of vectors (of different lengths)
    STree = OffsetArray(STree,0:n)      #convert so the indices are 0:n
    STree[0][1] = S                     #step 0 is in STree[0], element 1
    for i in 1:n                        #move forward in time
        STree[i][1:end-1] = u*STree[i-1] #most elements: up move from STree[i-1][1:end]
        STree[i][end] = d*STree[i-1][end] #last element: down move from STree[i-1][end]
    end
    return STree
end
```

BuildSTree

```

(S,m,y, $\sigma$ ,n) = (42,0.5,0.05,0.2,50)    #n=50 to facilitate plotting
(h,u,d,p) = CRRparams( $\sigma$ ,m,y,n)
STree = BuildSTree(S,n,u,d);

```

Showing the Tree for the Underlying Asset

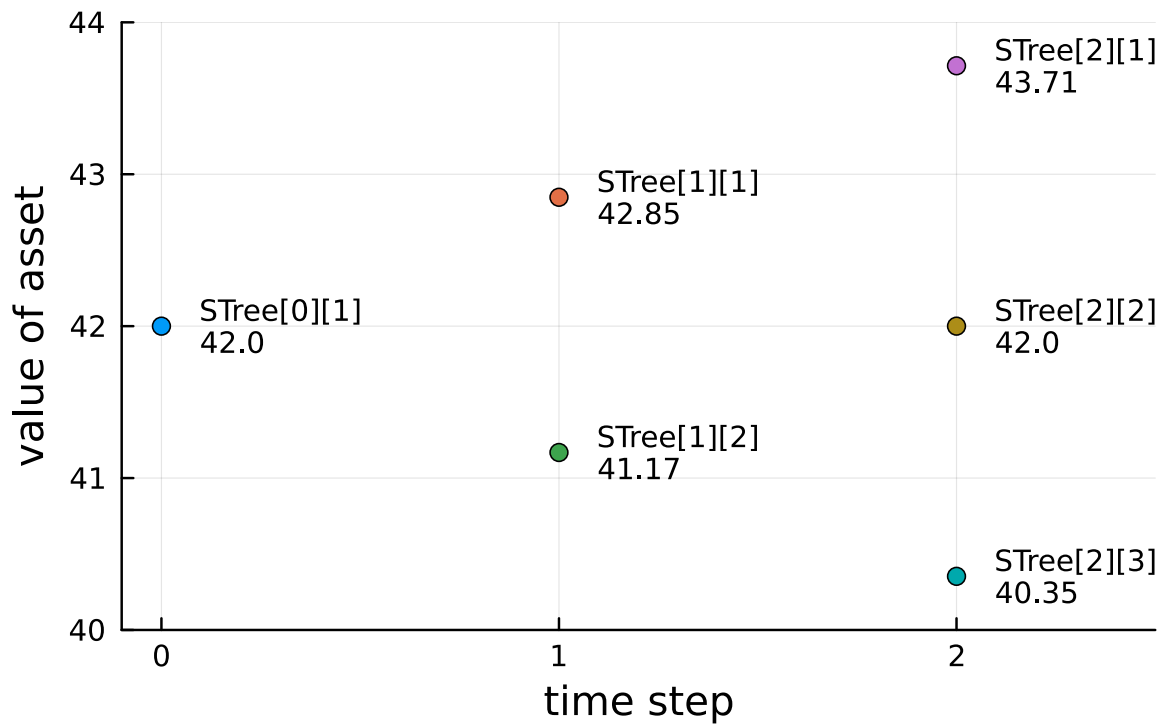
The next few cells illustrate the tree. Notice that `STree[i][1]` is the highest value of the underlying for time step `i`, and that `STree[i][2]` is the second highest, etc.

```

p1 = plot( legend = false,
           xlim = (-0.1,2.5),
           ylim = (40,44),
           xticks = 0:2,
           title = "Structure of STree (the first few nodes)",
           xlabel = "time step",
           ylabel = "value of asset" )
for i in 0:2, j in 1:length(STree[i])
    local txt1,txt2,txt
    txt1 = "STree[$i][$j]"
    txt2 = string(round(STree[i][j],digits=2))
    txt = text(string(txt1,"\n",txt2),8,:left)    #adding this info to plot
    scatter!( [i],[STree[i][j]],annotation = (i+0.1,STree[i][j],txt) )
end
display(p1)    #needed since plot() does not have data points to plot

```

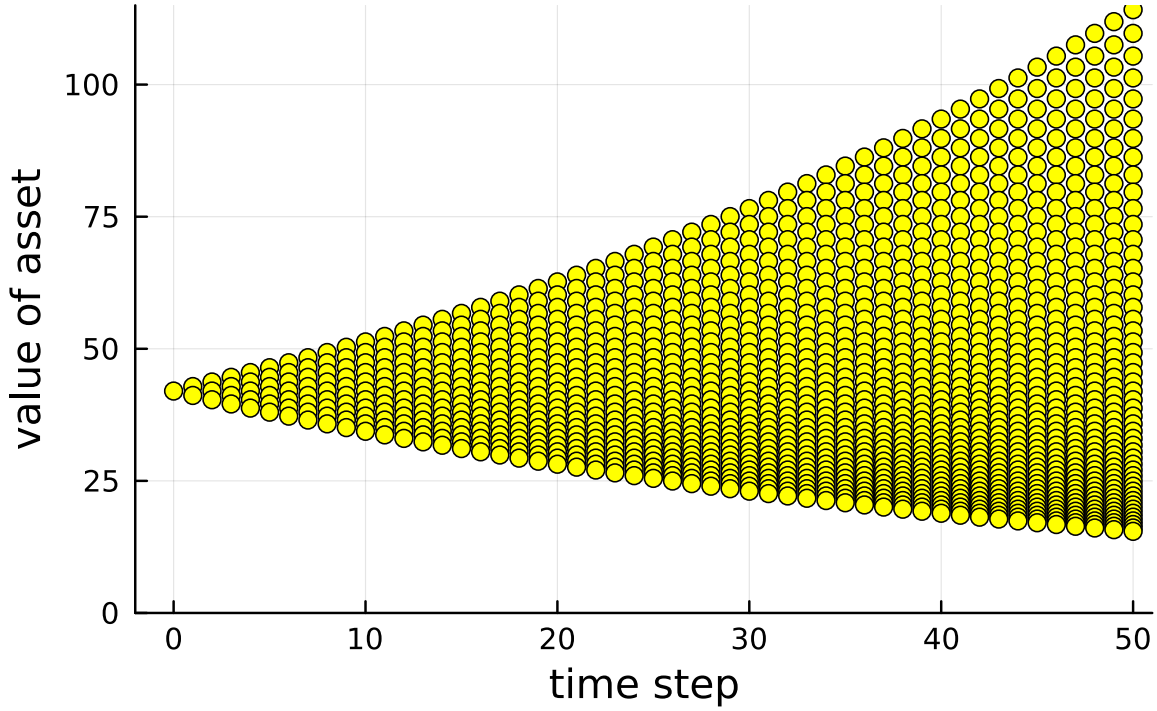
Structure of STree (the first few nodes)



```
p1 = scatter( [0.0],STree[0],
              markercolor = :yellow,
              xlim = (-2,n+1),
              ylim = (0,115),
              legend = false,
              title = "Nodes in STree",
              xlabel = "time step",
              ylabel = "value of asset" )

for i in 1:n
    scatter!(fill(i,i+1),STree[i],color=:yellow)
end
display(p1)
```

Nodes in STree



Calculating the Option Price

Let f_{ij} be the option price at time step i when the underlying price is S_{ij} . (We use S_{ij} as a shorthand notation, to avoid writing things like S_{tdd} .)

For a European call option, the call price at the *last time step* n is $f_{nj} = \max(0, S_{nj} - K)$ for each node j . Similarly, the put price is $f_{nj} = \max(0, K - S_{nj})$.

For all *earlier time steps*, the value for a European option is

$$f_{ij} = e^{-yh} [pf_{i+1,j} + (1-p)f_{i+1,j+1}].$$

In the code below, $f_{i+1,1}$ refers to the highest node (of the underlying) in time step $i+1$, $f_{i+1,2}$ to the second highest in the same time step, etc. Notice that we calculate the option price from the last time step (n) to the first (0).

A Remark on the Code

- Inside the function `EuOptionPrice()`, `Value = similar(STree)` is used to create a new vector (of vectors) called `Value` with the same structure as `STree`.
- The `;isPut=false` creates a keyword argument which defaults to `false`. To calculate a put price, call the function as `EuOptionPrice(STree,K,y,h,p;isPut=true)`.
- The code `Value[n] = isPut ? max.(0,K.-STree[n]) : max.(0,STree[n].-K)` is a short form of an `if...else...end`.
- `only(Pe[0])` creates a scalar from an array with one element. (Similar to `Pe[0][1]` but with error checking.)

European Options

```
"""
    EuOptionPrice(STree,K,y,h,p;isPut=false)

Calculate price of European option from binomial model

# Output
- 'Value:: Vector of vectors': option values at different nodes, same structure as STree
"""

function EuOptionPrice(STree,K,y,h,p;isPut=false)
    Value = similar(STree)           #tree for derivative, to fill
    n = length(STree) - 1           #number of steps in STree
    Value[n] = isPut ? max.(0,K.-STree[n]) : max.(0,STree[n].-K) #last time node
    for i in n-1:-1:0               #move backward in time
        Value[i] = exp(-y*h)*(p*Value[i+1][1:end-1] + (1-p)*Value[i+1][2:end])
    end                             #p*up + (1-p)*down, discount
    return Value
end
```

EuOptionPrice

```
K = 42.0                                #strike price

Pe = EuOptionPrice(STree,K,y,h,p;isPut=true)    #Pe[0] is a 1-element vector with the put price
Ce = EuOptionPrice(STree,K,y,h,p)

Ce_parity = only(Pe[0]) + S - exp(-m*y)*K      #put-call parity, only(Pe[0]) makes it a scalar
```

```
printblue("European option prices at K=$K and S=$S: ")
printmat([only(Pe[0]),only(Ce[0]),Ce_parity];rowNames=["put","call","call from parity"])
```

European option prices at K=42.0 and S=42:

put	1.844
call	2.881
call from parity	2.881

Routines for the Same Calculations, but Using Matrices to Store the Results (extra)

If you want to port the code to another language where a vector of vectors is tricky, then you might consider starting with the functions included below.

```
include("src/OptionsBopmMatrix.jl")
STreeM = BuildSTreeM(S,n,u,d)

println("first upper 4x4 block of the matrix:")
printmat(STreeM[1:4,1:4])

Pe2 = EuOptionPriceM(STreeM,K,y,h,p;isPut=true)
printlnPs("put price: ",Pe2[1,1])
```

first upper 4x4 block of the matrix:

42.000	42.848	43.714	44.597
NaN	41.168	42.000	42.848
NaN	NaN	40.353	41.168
NaN	NaN	NaN	39.554

put price: 1.844

American Options

The calculations are similar to those for a European option, except that the option value is

$$f_{ij} = \max(\text{value if exercised now}, \text{continuation value})$$

The *continuation value* has the same form as in the European case, and thus assumes that the option has not been exercised before the next period.

The *value of exercising now* is $S_{ij} - K$ for a call and $K - S_{ij}$ for a put.

```

"""
    AmOptionPrice(STree,K,y,h,p;isPut=false)

Calculate price of American option from binomial model

# Output
- `Value`: Vector of vectors: option values at different nodes, same structure as STree
- `Exerc`: Vector of vectors: true if early exercise at the node, same structure as STree

"""
function AmOptionPrice(STree,K,y,h,p;isPut=false)    #price of American option
    Value = similar(STree)                          #tree for derivative, to fill
    n      = length(STree) - 1
    Exerc = similar(Value,BitArray)                 #same structure as STree, but BitArray, empty
    Value[n] = isPut ? max.(0,K.-STree[n]) : max.(0,STree[n].-K) #last time node
    Exerc[n] = Value[n] .> 0                          #exercise
    for i in n-1:-1:0                                #move backward in time
        fa = exp(-y*h)*(p*Value[i+1][1:end-1] + (1-p)*Value[i+1][2:end])
        Value[i] = isPut ? max.(K.-STree[i],fa) : max.(STree[i].-K,fa) #put or call
        Exerc[i] = Value[i] .> fa                                     #true if early exercise
    end
    return Value, Exerc
end

```

AmOptionPrice

```

K = 42.0                                #strike price

(Pa,Exerc) = AmOptionPrice(STree,K,y,h,p;isPut=true)
(Ca,ExercC) = AmOptionPrice(STree,K,y,h,p)

printblue("Put and call prices at K=$K and S=$S: ")
xx = [ only(Pa[0]) only(Pe[0]);only(Ca[0]) only(Ce[0]) ]
printmat(xx,colNames=["American","European"],rowNames=["put","call"])

printred("When is the American option worth more? When the same?")

```

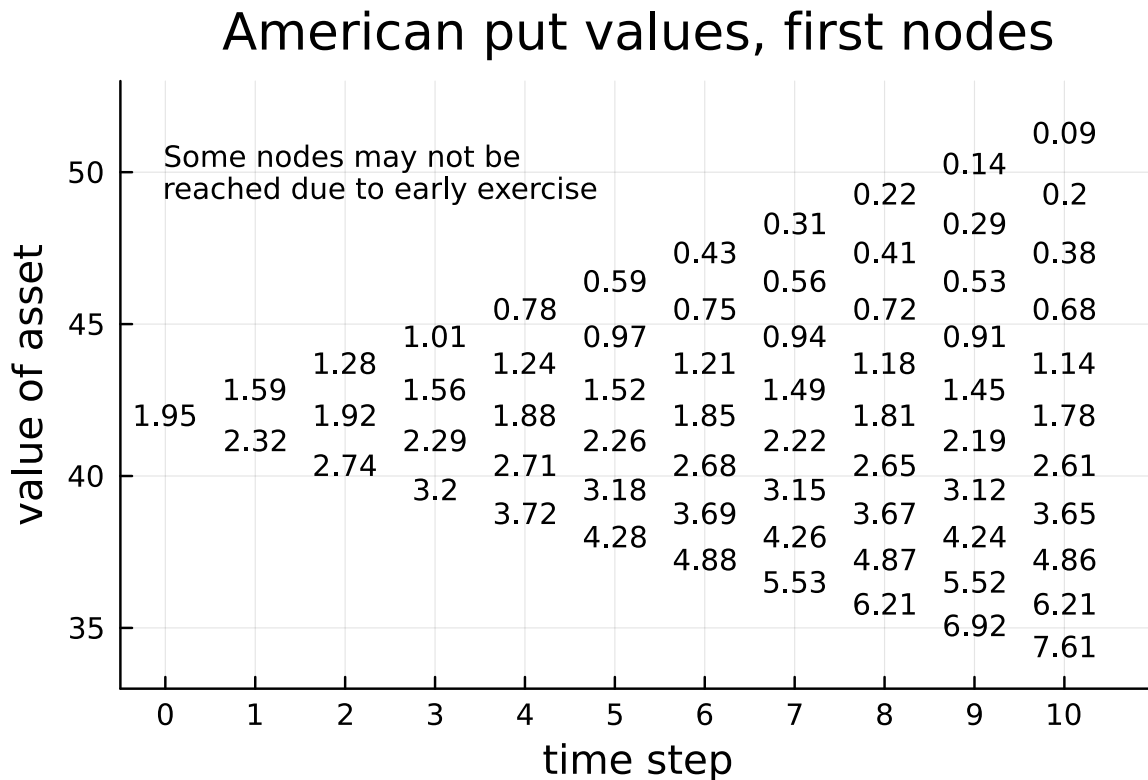
Put and call prices at K=42.0 and S=42:

	American	European
put	1.950	1.844
call	2.881	2.881

When is the American option worth more? When the same?

Plotting the Tree of American Option Values

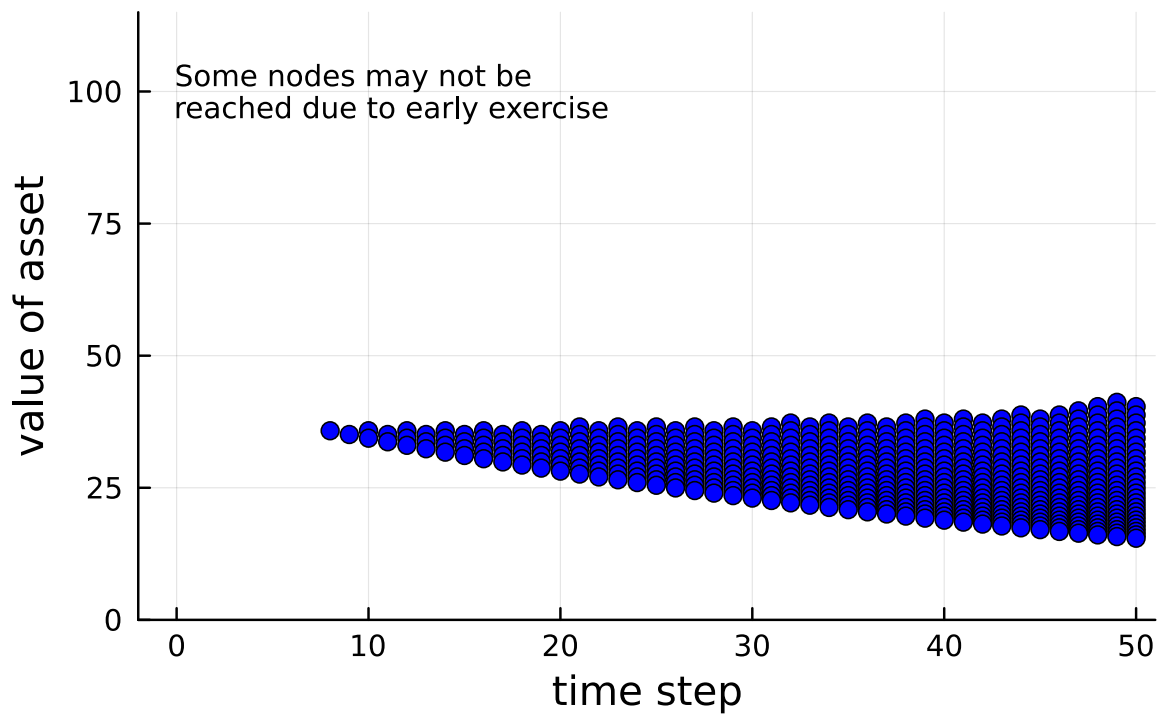
```
p1 = plot( xlim = (-0.5,11),
          ylim = (33,53),
          xticks = 0:10,
          legend = false,
          title = "American put values, first nodes",
          xlabel = "time step",
          ylabel = "value of asset",
          annotation = (0,50,text("Some nodes may not be\nreached due to early exercise",8,:left)))
for i in 0:10, j in 1:length(STree[i])
    annotate!(i,STree[i][j],text(string(round(Pa[i][j],digits=2)),8))
end
display(p1)
```



Plotting where Exercise Happens

```
p1 = plot( xlim = (-2,n+1),
           ylim = (0,115),
           legend = false,
           title = "Exercise nodes, put",
           xlabel = "time step",
           ylabel = "value of asset",
           annotation = (0,100,text("Some nodes may not be\nreached due to early exercise",8,:left))
for i in 0:n, j in 1:length(STree[i])
    if Exerc[i][j]
        scatter!([i],[STree[i][j]],markercolor=:blue)
    end
end
display(p1)
```

Exercise nodes, put

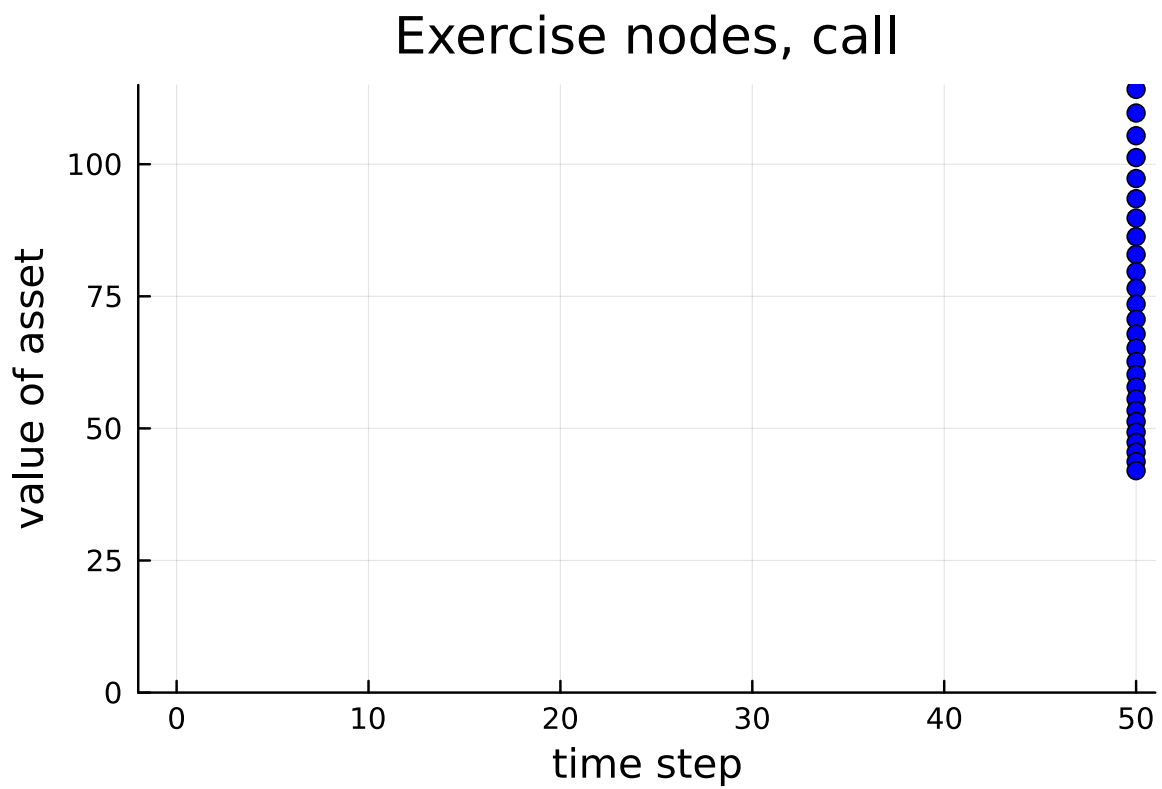


```
p1 = plot( xlim = (-2,n+1),
           ylim = (0,115),
```

```

        legend = false,
        title = "Exercise nodes, call",
        xlabel = "time step",
        ylabel = "value of asset")
for i in 0:n, j in 1:length(STree[i])
    if ExercC[i][j]
        scatter!([i],[STree[i][j]],markercolor=:blue)
    end
end
display(p1)

```



Options 3: The Black-Scholes Model

This notebook introduces the Black-Scholes option pricing model. It also discusses (a) implied volatility; (b) how to calculate the Black-Scholes model by numerical integration and (c) how the binomial model converges to the Black-Scholes model.

Load Packages and Extra Functions

```
using Printf, Distributions, Roots, QuadGK  
  
include("src/printmat.jl");
```

```
using Plots, LaTeXStrings  
default(size = (480,320),fmt = :png)
```

Black-Scholes

The Black-Scholes formula for a European call option on an (underlying) asset with a continuous dividend rate δ is

$C = e^{-\delta m} S \Phi(d_1) - e^{-ym} K \Phi(d_2)$, where

$$d_1 = \frac{\ln(S/K) + (y - \delta + \sigma^2/2)m}{\sigma\sqrt{m}} \quad \text{and} \quad d_2 = d_1 - \sigma\sqrt{m}$$

and where $\Phi(d)$ denotes the probability of $x \leq d$ when x has an $N(0, 1)$ distribution. In other words, $\Phi(d)$ is the cumulative distribution function of the $N(0, 1)$ distribution.

A Remark on the Code

- $\Phi(x) = \text{cdf}(\text{Normal}(0,1), x)$ is a one-line definition of a function.
- The `;isPut=false` creates a keyword argument which defaults to false which means that we calculate a call option price. To calculate a put option price, use the function as `OptionBlackSPs(...;isPut=true)`.

```
"""
     $\Phi(x)$ 
    Calculate  $\Pr(z \leq x)$  for  $N(0,1)$  variable  $z$ . Convenient short hand notation. Used in 'OptionBlackSPs()'
    """
 $\Phi(x) = \text{cdf}(\text{Normal}(0,1), x)$ 

"""
    Calculate Black-Scholes European call or put option price, continuous dividends of  $\delta$ 
    """
function OptionBlackSPs(S,K,m,y, $\sigma$ , $\delta=0$ ;isPut=false)
    d1 = ( log(S/K) + (y- $\delta+0.5*\sigma^2$ )*m ) / ( $\sigma*\text{sqrt}(m)$ )
    d2 = d1 -  $\sigma*\text{sqrt}(m)$ 
    c = exp(- $\delta*m$ )*S* $\Phi(d1)$  - K*exp(-y*m)* $\Phi(d2)$ 
    price = isPut ? c - exp(- $\delta*m$ )*S + exp(-y*m)*K : c
    return price
end
```

OptionBlackSPs

```
(S,K,m,y, $\sigma$ ) = (42,42,0.5,0.05,0.2)           #some parameter values

C = OptionBlackSPs(S,K,m,y, $\sigma$ )                #call
P = OptionBlackSPs(S,K,m,y, $\sigma$ ,0;isPut=true)    #put

 $\delta = 0.05$ 
C $\delta$  = OptionBlackSPs(S,K,m,y, $\sigma$ , $\delta$ )        #call, with dividends
P $\delta$  = OptionBlackSPs(S,K,m,y, $\sigma$ , $\delta$ ;isPut=true) #put

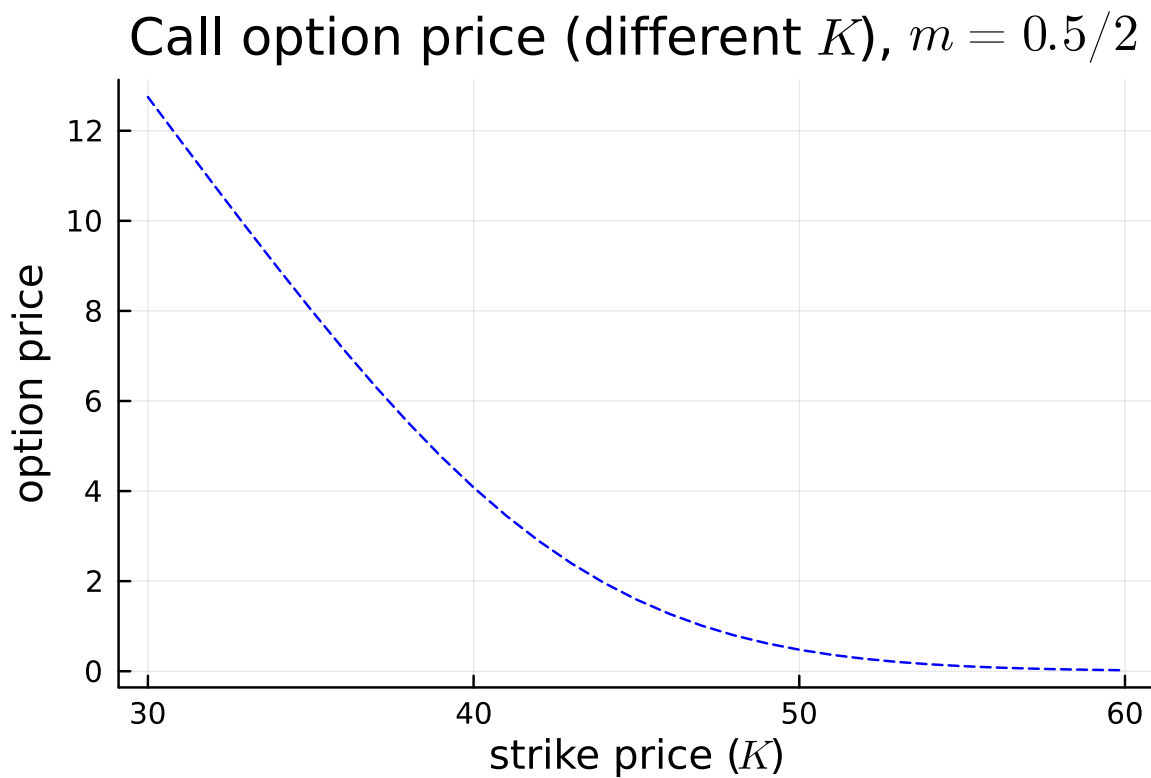
printblue("Option prices at S=$S and K=$K: ")
xx = [P P $\delta$ ;C C $\delta$ ]
printmat(xx;rowNames=["put","call"],colNames=["no dividends"," $\delta$ =$ $\delta$ "],width=15,colUnderlineQ=true)
```


Option prices at $S=42$ and $K=42$:

	no dividends	$\delta=0.05$
put	1.856	2.309
call	2.893	2.309

```
K_range = 30:60          #different strike prices
C_K      = OptionBlackSPs.(S,K_range,m,y, $\sigma$ )      #m = 1/4

p1 = plot( K_range,C_K,
           linecolor = :blue,
           linestyle = :dash,
           legend = false,
           xticks = 30:10:60,
           title = L"Call option price (different $K$), $m=0.5/2$",
           xlabel = L"strike price ($K$)",
           ylabel = "option price" )
display(p1)
```



How the Black-Scholes Depends on Volatility

The Black-Scholes option price is an increasing function of the volatility (σ), as illustrated below.

Implied Volatility

In the cell below we use an observed option price and solve the BS formula for σ . This is the “implied volatility,” which could be interpreted as the market belief about (annualised) standard deviation of the underlying until expiration of the option. We do this for several option prices: the Black-Scholes price and also higher and lower prices.

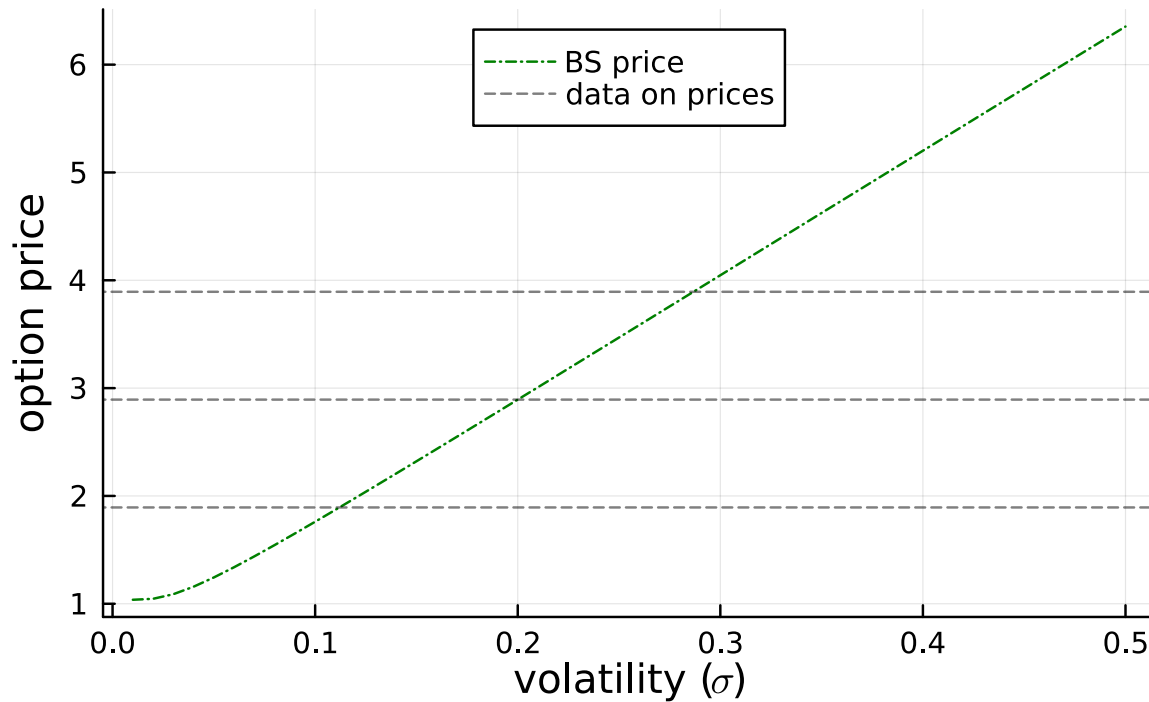
A Remark on the Code

- To solve for the implied volatility, the next cells use the [Roots.jl](#) package.
- We solve for the root of the anonymous function $\sigma \rightarrow \text{OptionBlackSPs}(S, K, m, y, \sigma) - C$ which finds σ so that the model based price from `OptionBlackSPs()` minus the actual price C equals 0. We loop over a few different values of C .
- `iv = [... for Ci in C_range]` creates a vector with one element per element in `C_range`.

```
σ_range = 0.01:0.01:0.5          #S,K,m,y are scalars, σ_range is a vector
C_σ      = OptionBlackSPs.(S,K,m,y,σ_range)    #at different σ values
C_range = [C-1,C,C+1]            #cheaper,B-S price,expensive

p1 = plot( σ_range,C_σ,
           linecolor = :green,
           linestyle = :dashdot,
           label = "BS price",
           title = L"Call option price (different $\sigma$)",
           xlabel = L"volatility ($\sigma$)",
           ylabel = "option price",
           legend = :top )
hline!( C_range,linetype=:hline,linecolor=:black,line=(:dash,0.5),label="data on prices")
display(p1)
```

Call option price (different σ)



```
iv = [find_zero( $\sigma \rightarrow$ OptionBlackSPs(S,K,m,y, $\sigma$ )-Ci, (0,5)) for Ci in C_range] #create a vector of results
#for value in C_range

printblue("implied volatility:")
printmat(iv;rowNames=["Cheap option","Benchmark","Expensive option"])

printred("Compare the results with the previous graph\n")
```

implied volatility:

Cheap option	0.112
Benchmark	0.200
Expensive option	0.287

Compare the results with the previous graph

BS from an Explicit Integration

The price of a European call option is

$$C = e^{-ym} E^* \max(0, S_m - K),$$

which can be written

$$C = e^{-ym} \int_K^\infty \max(0, S_m - K) f^*(S_m) dS_m,$$

where $f^*(S_m)$ is the risk neutral density function of the asset price at expiration (S_m). The integration starts at $S_m = K$ since $\max(0, S_m - K) = 0$ for $S_m < K$. See the figure below.

In the Black-Scholes model, the risk neutral distribution of $\ln S_m$ is

$$\ln S_m \sim^* N(\ln S + my - m\sigma^2/2, m\sigma^2),$$

where S is the current asset price. This means that $f^*(S_m)$ is the pdf of a lognormally distributed variable. Notice that this is the pdf of S_m , not its logarithm.

A Remark on the Code

- The `LogNormal` in the `Distributions.jl` package wants the mean and standard deviation of $\ln S_m$ as inputs, but calculates the pdf of S_m .
- The numerical integration is done by the [QuadGK.jl](#) package.
- The integration is of the anonymous function $x \rightarrow \text{BSintegrand}(x, S, K, y, m, \sigma)$ over the interval $[K, \text{Inf}]$. Here, x represents possible values of S_m .

```

"""
BSintegrand(Sm,S,K,y,m,σ)

Constructs the integrand for the Black-Scholes call price
"""

function BSintegrand(Sm,S,K,y,m,σ)
    μ = log(S) + m*y - m*σ^2/2          #"mean"
    λ = sqrt(m)*σ                       #"std"
    f = pdf(LogNormal(μ,λ),Sm)          #log-normal pdf(mean,std)
    z = exp(-y*m)*max(0,Sm-K)*f
    return z
end

```

BSintegrand (generic function with 1 method)

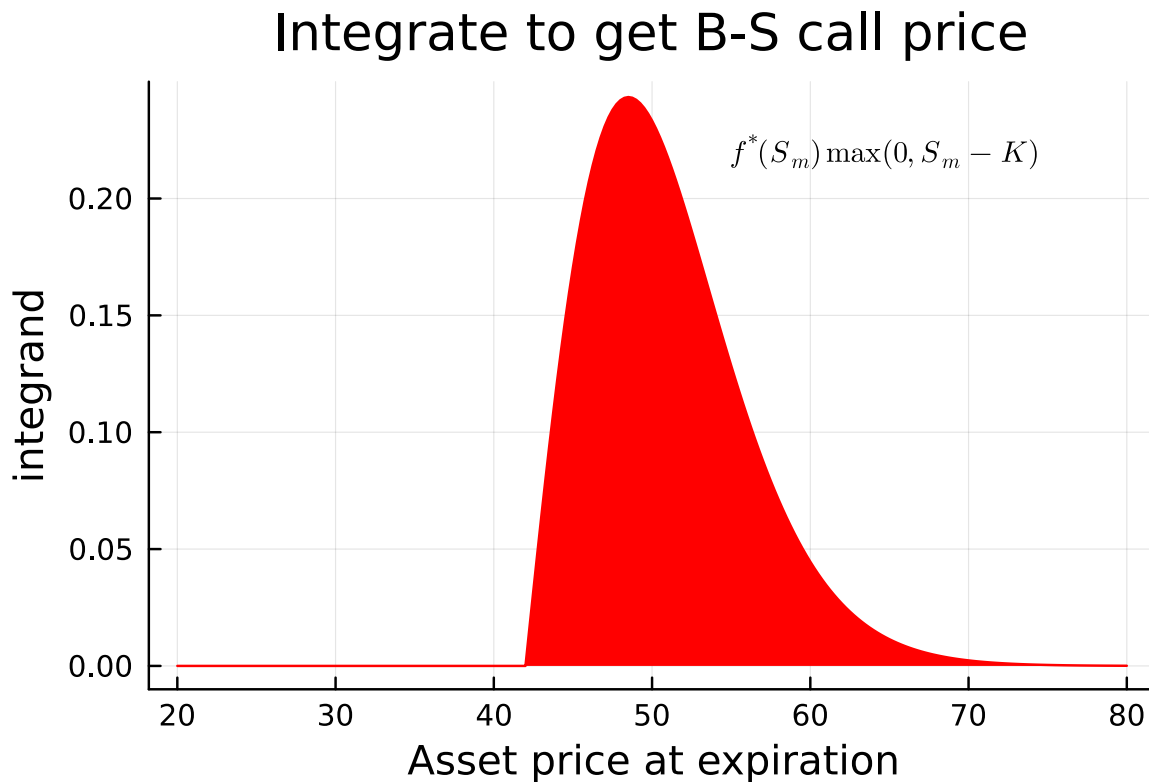
```

Sm_range = 20:0.25:80      #possible outcomes of underlying price

#txt = text("pdf * payoff",8,:left)
txt = text(L"f^\ast(S_m) \max(0,S_m-K)",8,:left)

p1 = plot( Sm_range,BSintegrand.(Sm_range,S,K,y,m,\sigma),
           linecolor = :red,
           ylim = (-0.01,0.25),
           fill = (0,:red),
           legend = false,
           title = "Integrate to get B-S call price",
           xlabel = "Asset price at expiration",
           ylabel = "integrand",
           annotation = (55,0.22,txt) )
display(p1)

```



```

C1, = QuadGK.quadgk(x→BSintegrand(x,S,K,y,m,\sigma),K,Inf)  #numerical integration over (K,Inf)

```

```
printblue("Call option price:")
printmat([C1,C],rowNames=["from numerical integration","from BS formula"])
```

```
Call option price:
from numerical integration    2.893
from BS formula              2.893
```

Convergence of BOPM to BS

The next few cells calculate the option price according to binomial model with a CRR calibration where

$$u = e^{\sigma\sqrt{h}}, d = e^{-\sigma\sqrt{h}} \text{ and } p = \frac{e^{yh}-d}{u-d}.$$

This is done repeatedly, using more and more time steps (n) with $h = m/n$ where m is the fixed time to expiration.

The file included below contains, among other things, the functions BuildSTree() and EuOptionPrice() from the chapter on the binomial model.

```
using OffsetArrays
include("src/OptionsCalculations.jl");
```

```
 #(S,K,m,y,σ) = (42,42,0.5,0.05,0.2)      #these parameters were defined before
```

```
nMax = 50
```

```
C_bopm = fill(NaN,nMax)
```

```
for n in 1:nMax
```

```
    #calculate option price nMax times
```

```
    #local h, u, d, p, STree, Ce
```

```
    #local/global is needed in script
```

```
    (h,u,d,p) = CRRparams(σ,m,y,n)
```

```
    STree = BuildSTree(S,n,u,d)
```

```
    Ce = EuOptionPrice(STree,K,y,h,p)
```

```
    C_bopm[n] = only(Ce[0])
```

```
    #pick out the call price at the starting node, make it scalar
```

```
end
```

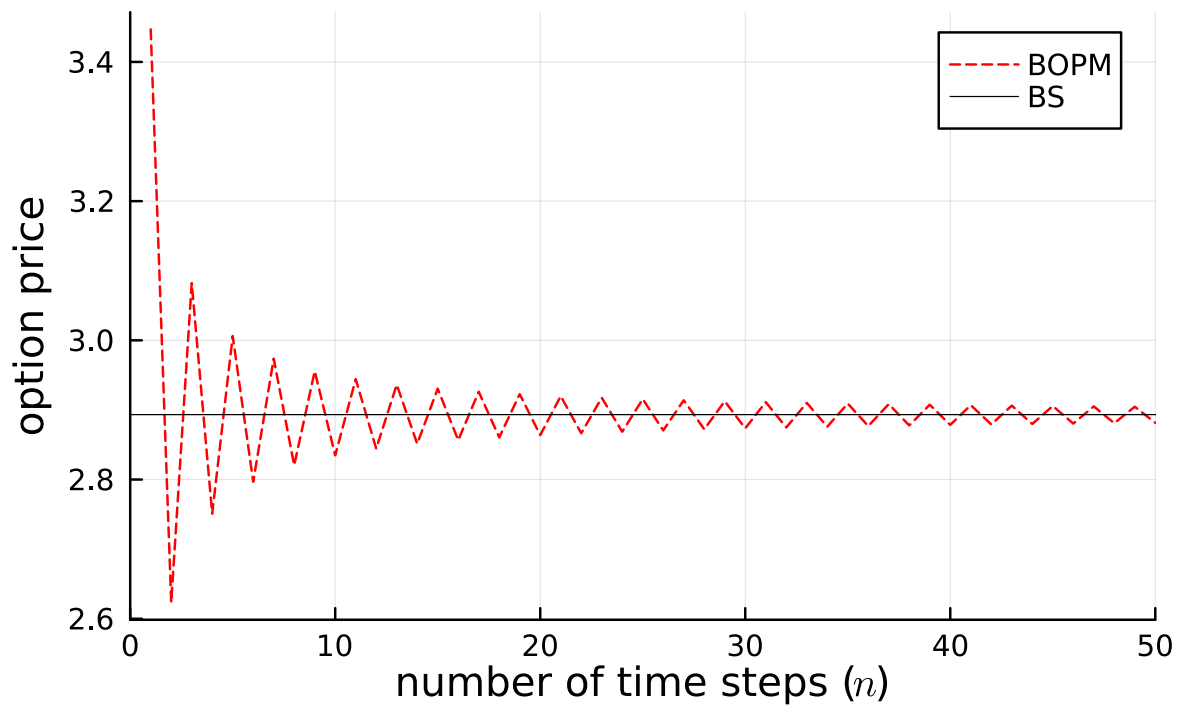
```
p1 = plot( 1:nMax,C_bopm,
           linecolor = :red,
           linestyle = :dash,
           linewidth = 1,
           xlims = [0,nMax],
```

```

label = "BOPM",
title = "BOPM and BS call option price",
xlabel = L"number of time steps ($n$)",
ylabel = "option price",
annotation = (170,2.7,text(L"h = m/n",8)) )
hline!([C],linecolor=:black,line=:solid,linewidth=0.5,label="BS")
display(p1)

```

BOPM and BS call option price



Options 4: Hedging Options

This notebook illustrates how to hedge an option by holding a position in the underlying asset (delta hedging).

Load Packages and Extra Functions

```
using Printf, Distributions  
  
include("src/printmat.jl");
```

```
using Plots, LaTeXStrings  
default(size = (480,320),fmt = :png)
```

A First-Order Approximation of the Option Price Change

“Delta hedging” is based on the idea that we can approximate the change in the option price by

$$C_{t+h} - C_t \approx \Delta (S_{t+h} - S_t),$$

where Δ is the derivative of the call option price wrt. the underlying asset price, called the *delta*. (It does *not* indicate a first difference.)

In the Black-Scholes model, the delta of a call option is

$$\Delta_c = \frac{\partial C}{\partial S} = e^{-\delta m} \Phi(d_1),$$

where d_1 is the usual term in Black-Scholes and δ is the continuous dividend rate (possibly 0).

Similarly, the delta of a put option is

$$\Delta_p = \Delta_c - e^{-\delta m}.$$

The file included in the next cell contains the functions $\Phi()$ and OptionBlackSPs() from the chapter on the Black-Scholes model.

The subsequent cell defines a function for the Δ of the Black-Scholes model.

```
include("src/OptionsCalculations.jl");

"""
Calculate the Black-Scholes delta
"""
function OptionDelta(S,K,m,y,σ,δ=0;isPut=false)
    d1 = ( log(S/K) + (y-δ+0.5*σ^2)*m ) / (σ*sqrt(m))
    d2 = d1 - σ*sqrt(m)
    Δ = isPut ? exp(-δ*m)*(Φ(d1)-1) : exp(-δ*m)*Φ(d1)    #put or call
    return Δ
end
```

OptionDelta

```
(S,K,m,y,σ) = (42,42,0.5,0.05,0.2)

Δ_c = OptionDelta(S,K,m,y,σ)           #call
Δ_p = OptionDelta(S,K,m,y,σ,0;isPut=true) #put

printblue("Δ when the underlying asset has no dividends:\n")
printmat([Δ_c Δ_p (Δ_c-Δ_p)];colNames=["call","put","difference"],width=12,colUnderlineQ=true)
```

Δ when the underlying asset has no dividends:

call	put	difference
----	---	-----
0.598	-0.402	1.000

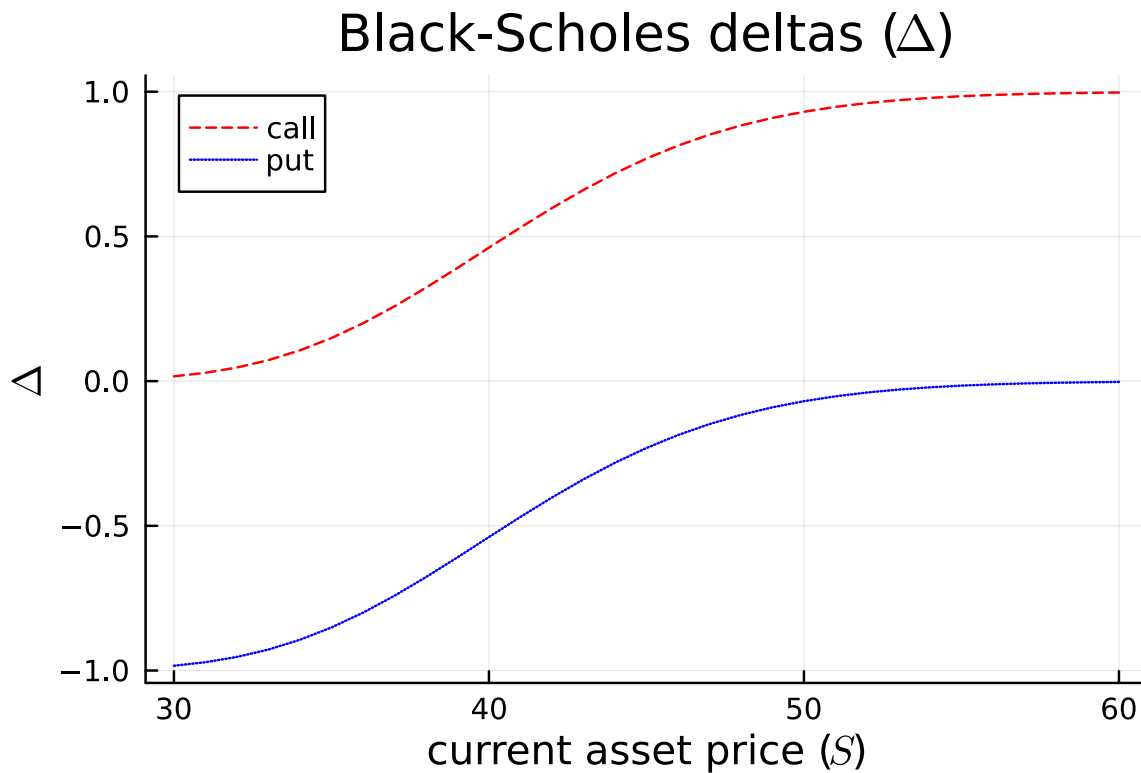
```
S_range = 30:60           #different spot prices
Δ_c_S   = OptionDelta.(S_range,K,m,y,σ)
Δ_p_S   = OptionDelta.(S_range,K,m,y,σ;isPut=true)

p1 = plot( S_range,[Δ_c_S Δ_p_S],
            linecolor = [:red :blue],
            linestyle = [:dash :dot],
            label = ["call" "put"],
```

```

title = L"Black-Scholes deltas ($\Delta$)",
xlabel = L"current asset price ($S$)",
ylabel = L"\Delta" )
display(p1)

```



Hedging an Option

The example below shows how delta hedging works for a European call option when the price of the underlying asset changes (from 42 on day 0 to 43 on day 1). For simplicity, we assume that the Black-Scholes model is a good description of how the option price is set.

```

( $S_0, S_1, K, m, y, \sigma$ ) = (42, 43, 42, 0.5, 0.05, 0.2) #prices before, after, parameters

 $C_0$  = OptionBlackSPs( $S_0, K, m, y, \sigma$ ) #option price at  $S_0$ 
 $\Delta_0$  = OptionDelta( $S_0, K, m, y, \sigma$ ) #Delta at  $S_0$ 
 $M_0$  =  $C_0 - \Delta_0 * S_0$  #on money market account

```

```

C1 = OptionBlackSPs(S1,K,m-1/252,y,σ) #option price at S1 (it's one day later)
dC = C1 - C0 #change of option value
dV = Δ0*(S1-S0) - (C1-C0) #change of hedge portfolio value

xy = [S0,Δ0,C0,M0,S1,C1,dC,dV]
printmat(xy;rowNames=["S0","Δ0","C0","M0","S1","C1","dC","dV"],prec=2)

printred("\nV changes much less in value than the option, that is, abs(dV) < abs(dC),
so the hedge helps")

```

```

S0      42.00
Δ0      0.60
C0      2.89
M0     -22.21
S1      43.00
C1      3.51
dC      0.62
dV     -0.02

```

V changes much less in value than the option, that is, $\text{abs}(dV) < \text{abs}(dC)$, so the hedge helps

Hedging an Option Portfolio

In this case, we have issued nc call options and np put options with strike K and want to know how many units of the underlying asset that we need in order to be hedged. The delta of this portfolio is $nc \cdot \Delta(\text{call}) + np \cdot \Delta(\text{put})$.

The example uses $(nc,np) = (3,-2)$. Change to $(1,1.5)$ to see what happens. (Maybe the hedge does not work so well in this case...although that is not evaluated here.)

```

(S0,K,m,y,σ) = (42,42,0.5,0.05,0.2)

(nc,np) = (3,-2)
#(nc,np) = (1,1.5) #try this too

Δ_call = OptionDelta.(S0,K,m,y,σ) #Delta of call option
Δ_put  = OptionDelta.(S0,K,m,y,σ,0;isPut=true) #Delta of put option
Δ      = nc*Δ_call + np*Δ_put

```

```
xy = [ $\Delta$ _call, $\Delta$ _put, $\Delta$ ]  
printmat(xy;rowNames=[" $\Delta$  of call"," $\Delta$  of put"," $\Delta$  of option portfolio"])  
printred("We need to buy $(round( $\Delta$ ,digits=3)) units of the underlying")
```

Δ of call	0.598
Δ of put	-0.402
Δ of option portfolio	2.598

We need to buy 2.598 units of the underlying

Basic Matrix Algebra

This notebook presents some basic linear algebra and some matrix reshuffling/computations often needed in finance and econometrics.

```
using Printf, LinearAlgebra  
  
include("src/printmat.jl");
```

Adding and Multiplying: An Array and a Scalar

With an array A (for instance, a vector or a matrix, but also higher-dimensional arrays) and a scalar c , do

1. $A * c$ (textbook: Ac) to multiply each element of A by c
2. $A .+ c$ (textbook: $A + cJ$, where J is an array of ones) to add c to each element of A , and similarly $A .- c$ (textbook: $A - cJ$). Notice the dot.

Watch out when the number comes first: $2.*A$ is not allowed since it is ambiguous. However, $2.*A$ and $2 .+ A$ both work.

```
A = [1 3; 3 4]  
c = 10  
  
printblue("A:")  
printmat(A)  
printblue("c:")  
printmat(c)  
  
printblue("A*c:")  
printmat(A*c)
```

```
printblue("A .+ c:")
printmat(A .+ c)      #notice the dot in .+
```

```
A:
    1      3
    3      4
```

```
c:
    10
```

```
A*c:
    10      30
    30      40
```

```
A .+ c:
    11      13
    13      14
```

Adding and Multiplying Two Arrays

With two arrays of the same dimensions (A and B), do

$A+B$ (textbook: $A + B$) to add them (element by element), and similarly $A-B$ (textbook: $A - B$).

To multiply arrays (A and B) of conformable dimensions: $A*B$ (textbook: AB).

```
A = [1 3;3 4]      #A and B are 2x2 matrices
B = [1 2;3 -2]
printblue("A:")
printmat(A)
printblue("B:")
printmat(B)

printblue("A+B:")
printmat(A+B)

printblue("A*B:")
printmat(A*B)
```

```
A:
    1      3
```

	3	4
B:		
	1	2
	3	-2
A+B:		
	2	5
	6	2
A*B:		
	10	-4
	15	-2

Transpose

You can transpose a numerical vector/matrix A by A' .

Notice that (in Julia) A and $B = A'$ share the same elements (changing one changes the other). If you want an independent copy, use $B = \text{permutedims}(A)$ or perhaps $B = \text{copy}(A')$.

For an array of other elements (for instance, strings), use $\text{permutedims}(A)$ to swap the dimensions.

```
A = [1 2 3; 4 5 6]
printblue("A: ")
printmat(A)
printblue("A': ")
printmat(A')
```

A:			
	1	2	3
	4	5	6
A':			
	1	4	
	2	5	
	3	6	

Vectors: Inner and Outer Products

There are several different ways to think about a vector in applied mathematics: as a $K \times 1$ matrix (a column vector), a $1 \times K$ matrix (a row vector) or just a flat K vector. Julia uses flat vectors but they are mostly interchangeable with column vectors.

The inner product of two (column) vectors with K elements is calculated as $x'z$ or `dot(x,y)` (textbook: $x'z$ or $x \cdot z$) to get a scalar. (The dot is obtained by `\cdot` + TAB, but this is sometimes hard to distinguish from or things like $x \cdot z$.)

In contrast, the outer product of two (column) vectors with K elements is calculated as xz' (textbook: xz') to get a $K \times K$ matrix.

```
x = [10,11]          #[10;11] gives the same
z = [2,5]
printblue("x and z:")
printmat(x,z)

printblue("x'z: ")
printlnPs(x'z)        #dot(x,z) gives the same

printblue("x*z':")
printmat(x*z')
```

x and z:

10	2
11	5

x'z:

75

x*z':

20	50
22	55

Vectors: Quadratic Forms

A quadratic form (A is an $n \times n$ matrix and x is an n vector): $x'A*x$ (textbook: $x'Ax$) to get a scalar. There is also the function `dot(x,A,x)`.


```

A = [1 3;3 4]
x = [10,11]

printblue("x:")
printmat(x)
printblue("A:")
printmat(A)

printblue("x'A*x: ")
printlnPs(x'A*x)      #or dot(x,A,x)

```

x:

```

10
11

```

A:

```

1      3
3      4

```

x'A*x:

```

1244

```

Kronecker Product

The Kronecker product of two matrices

$$A \otimes B = \begin{bmatrix} a_{11}B & \cdots & a_{1n}B \\ \vdots & & \vdots \\ a_{m1}B & \cdots & a_{mn}B \end{bmatrix}$$

is calculated by using `kron()`

```

A = [1 3;2 4]
B = [10 11]
z = kron(A,B)

printblue("A:")
printmat(A)
printblue("B:")
printmat(B)

```

```
printblue("kron(A,B):")
printmat(z)
```

A:

1	3
2	4

B:

10	11
----	----

kron(A,B):

10	11	30	33
20	22	40	44

The Identity Matrix

The identity matrix I_n can often be represented by `I` and then Julia will compare with the surrounding code to create the right dimension. For instance, if `A` is a square matrix, then `I + A` and `[A I]` both work. Notice that you need to do using `LinearAlgebra` before `I` works.

If you still need to specify the dimension, then use `1I(3)` or `1I[1:3,1:3]`.

You can create a basis vector as `1I[1:3,2]`. This 3-vector is filled with zeros, except that element 2 is 1.

```
printblue("I*A and A*I")
printmat(I*A,A*I)
```

```
printblue("I + A")
printmat(I + A)
```

```
printblue("1I[1:3,1:3]")
printmat(1I[1:3,1:3])
```

```
printblue("1I[1:3,2]")
printmat(1I[1:3,2])
```

I*A and A*I

1	3	1	3
2	4	2	4

$I + A$

2	3
2	5

$1I[1:3,1:3]$

1	0	0
0	1	0
0	0	1

$1I[1:3,2]$

0
1
0

Matrix Inverse

The rank of a matrix is the number of linearly independent columns.

A matrix inverse of an $n \times n$ matrix A :

$\text{inv}(A)$ or A^{-1} (textbook: A^{-1})

The inverse is such that $AA^{-1} = I$ and $A^{-1}A = I$, where I is the identity matrix (ones along the main diagonal, zeros elsewhere).

```
A = [1 3; 3 4]
printblue("A:")
printmat(A)

printblue("rank(A):")
printlnPs(rank(A), "\n")

printblue("inv(A):")
printmat(inv(A))

printblue("inv(A)*A:")
printmat(inv(A)*A)
```

A:

1	3
3	4

rank(A):

2

```
inv(A):  
  -0.800    0.600  
  0.600   -0.200
```

```
inv(A)*A:  
  1.000   -0.000  
  0.000    1.000
```

Solving Systems of Linear Equations

If you have the system $Ab = y$, where A is a matrix and b and y are vectors, then we can solve as $b = \text{inv}(A)*y$ or $b = A \backslash y$. The latter is typically more robust.

```
y = [10,11]  
  
b1 = inv(A)*y  
b2 = A \ y  
  
printmat(b1,b2,colNames=["sol 1","sol 2"])  
  
printblue("checking the solution: is A*b1-y=0?")  
printmat(A*b1-y)
```

```
sol 1    sol 2  
-1.400   -1.400  
 3.800    3.800
```

```
checking the solution: is A*b1-y=0?  
 0.000  
 0.000
```

Eigenvalues and Eigenvectors

using the `svd()` function, since it automatically sorts in descending order.

```

Ω = [5 2;
     2 2]

F = svd(Ω)
λ = F.S      #eigenvalues
W = F.U      #vectors

printmat(λ,W;colNames=["λ","W1","W2"])

tst = W*diagm(λ)W'      #does it recreate Ω?
printmat(tst-Ω)

```

λ	W ₁	W ₂
6.000	-0.894	-0.447
1.000	-0.447	0.894
0.000	0.000	
0.000	0.000	

Vectors: Extracting Vectors from Matrices

Notice that $A[1,:]$ and $A[:,1]$ both give flat vectors. In case you want a row vector use $A[1:1,:]$.

```
A[1,:]
```

2-element Vector{Int64}:

```
1
3
```

Review of Statistics

This notebook shows some basic statistics needed for this course.

The notebook uses the `Statistics` package (a standard library, built-in) for descriptive statistics (averages, autocorrelations, etc) and the [Distributions.jl](#) package for statistical distributions (pdf, cdf, etc).

The `DelimitedFiles` and `LinearAlgebra` (standard libraries) are used to read a csv file with data and some basic matrix manipulations.

Load Packages and Extra Functions

```
using Printf, Statistics, DelimitedFiles, LinearAlgebra, Distributions

include("src/Ols.jl")
include("src/printmat.jl");
```

```
using Plots, LaTeXStrings          #packages for plotting and LaTeX
default(size = (480,320),fmt = :png) #or :svg
```

Distributions

Probability Density Function (pdf)

The cells below calculate and plot pdfs and cdfs of some distributions often used in the lecture notes. The `Distributions.jl` package has many more distributions.

A Remark on the Code

- Notice that the Distributions.jl package wants $\text{Normal}(\mu, \sigma)$, where σ is the standard deviation. However, the notation in the lecture notes is $N(\mu, \sigma^2)$. For instance, $N(0, 2)$ from the lectures is coded as $\text{Normal}(0, \text{sqrt}(2))$.
- $\text{pdf}(\text{Normal}(0, 1), x)$ calculates the pdf of a standard normal variable at each value in the array x . Notice the dot ($.$).

```
x = -4:0.1:4
pdf1a = pdf.(Normal(0,1),x)           #Normal
pdf1b = pdf.(Normal(1,1),x)
pdf1c = pdf.(Normal(0,sqrt(2)),x)

pdf2 = pdf.(LogNormal(0.25,sqrt(0.75)),x)   #LogNormal

(n,p,a,b) = (11,0.55,0.35,-0.25)           #Binomial
z = 0:n
z2 = z*a + (n-z)*b                         #shifted binomial, 1→a, 0→b
pdf3 = pdf.(Binomial(n,p),z)

p1 = plot( x,hcat(pdf1a,pdf1b,pdf1c),
            linecolor = [:blue :red :black],
            linestyle = [:solid :dash :dot],
            linewidth = [1 1 2],
            legend = nothing,
            ylims = (0,0.4),
            title = "Normal pdfs",
            xlabel = "x")

p2 = plot( x,pdf2,
            linecolor = :red,
            legend = nothing,
            ylims = (0,0.6),
            title = "Lognormal pdf",
            xlabel = "x")

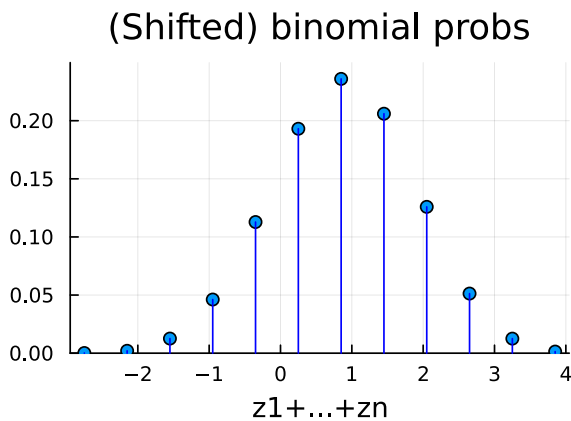
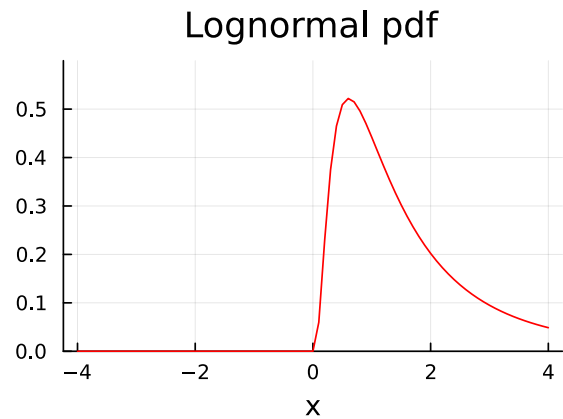
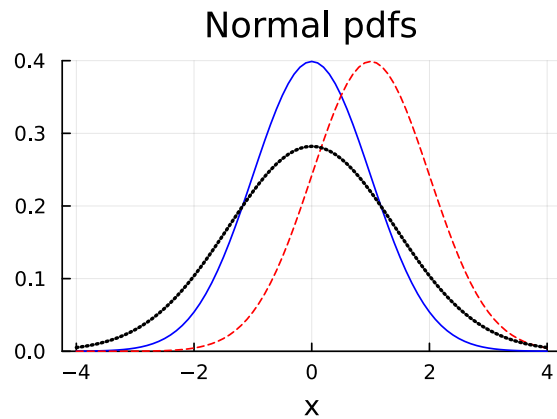
p3 = plot( z2,pdf3,
            linecolor = :blue,
            legend = nothing,
            seriestype=:stem,
            marker=:circle,
```

```

        ylims = (0,0.25),
        title = "(Shifted) binomial probs",
        xlabel = "z1+...+zn")

pAll = plot( p1,p2,p3,                                #combine the subplots
            layout = @layout[a a;a _],
            size = (700,500) )
display(pAll)

```



```

x  = -3:0.1:3
xb = x[x.<=0.5]                                #pick out x values <= 0.5

pdfx  = pdf.(Normal(0,1),x)                    #calculate the pdf of a N(0,1) variable
pdfxb = pdf.(Normal(0,1),xb)                  #colour the area below this curve
Prb = cdf(Normal(0,1),0.5)
printlnPs("Pr(x<=0.5) ",Prb)

```

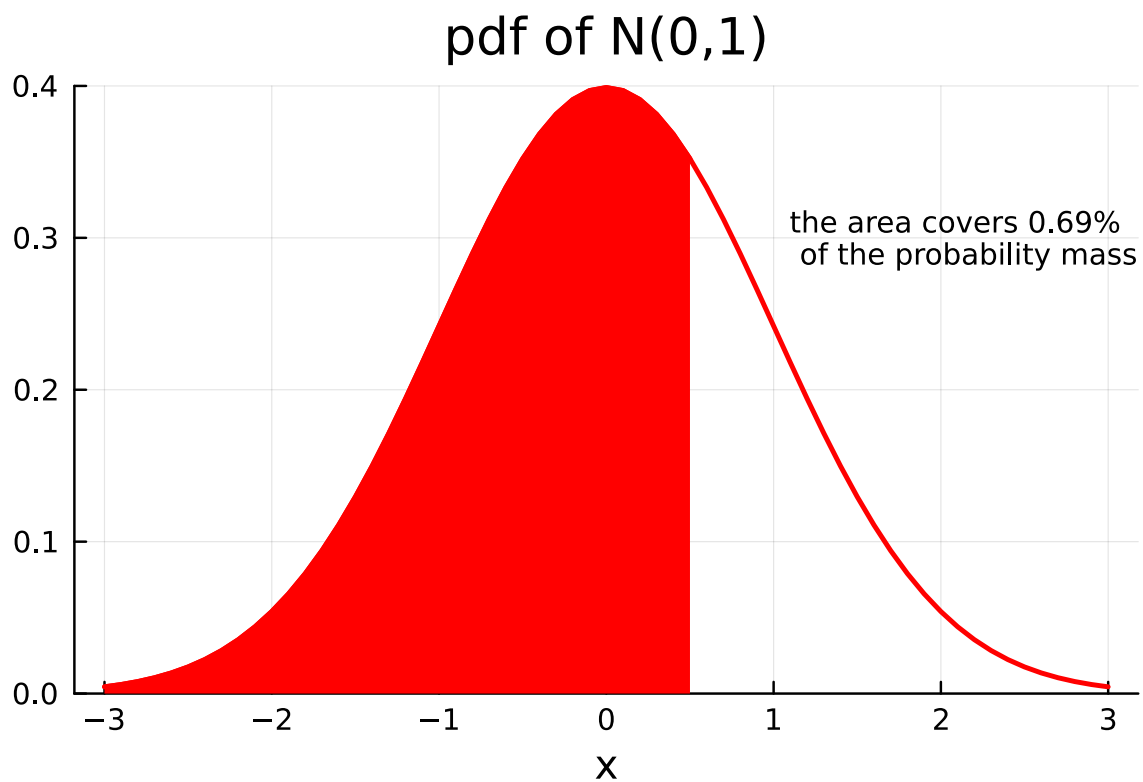


```

p1 = plot( x,pdfx,                                     #plot pdf
           linecolor = :red,
           linewidth = 2,
           legend = nothing,
           ylims = (0,0.4),
           title = "pdf of N(0,1)",
           xlabel = "x",
           annotation = (1.1,0.3,text("the area covers $(round(Prb,digits=2))%\n of the probability
plot!(xb,pdfxb,linecolor=:red,linewidth=2,legend=nothing,fill=(0,:red))    #plot area under pdf
display(p1)

```

Pr($x \leq 0.5$) 0.691



```
printlnPs("5% quantile of N(0,1): ", quantile(Normal(0,1),0.05))
```

5% quantile of N(0,1): -1.645

Expected Values and Variances

The random variable used in the example below can only take two values in the vector $x = [x_1, x_2]$, with probabilities in $p = [p_1, 1-p_1]$. (π is already defined in one of the packages.)

We define a simple function $E(x, p)$ to calculate an expectation.

```
"""
    E(x,p)

Calculate the expectation from vectors of outcomes and their probabilities.
"""
E(x,p) = sum(p.*x)      #or p'x
```

E

```
x = [9.5,11]
p = [0.4,0.6]          #probabilities

μ = E(x,p)
printlnPs("μ: ",μ)

σ² = E((x.-μ).^2,p)
printlnPs("σ²: ",σ²)

printlnPs("\nCheck the variance against the theoretical result:", p[1]*p[2]*(x[2]-x[1])^2)
```

```
μ:      10.400
σ²:      0.540
```

Check the variance against the theoretical result: 0.540

Expected Value of a Function and Its Derivative

```
Elnx = E(log.(x),p)      #expected value of log(x)
println("log(x) and its expected value")
Edlnx = E(1.0./x,p)      #derivative of log(x) is 1/x

xut = hcat([x;μ],[log.(x);Elnx],[1.0./x;Edlnx])
printmat(xut;rowNames=["state 1","state 2","expected value"],colNames=["x","log(x)","dlog(x)/dx"],w
```

log(x) and its expected value

	x	log(x)	dlog(x)/dx
state 1	9.500	2.251	0.105
state 2	11.000	2.398	0.091
expected value	10.400	2.339	0.097

Expected Value and Variance of Linear Combinations

```
 $\mu$  = [11.5, 9.5, 6]/100          #expected returns for 3 assets
 $\Sigma$  = [166 34 58;             #covariance matrix
        34 64 4;
        58 4 100]/100^2
Rf = 0.03                        #riskfree return
```

0.03

```
w = [0.25,0.68,0.07]           #portfolio weights summing to 1
```

3-element Vector{Float64}:

0.25
0.68
0.07

```
ERp = w'* $\mu$ 
VarRp = w'* $\Sigma$ *w

xut = [ERp,VarRp,sqrt(VarRp)]
printmat(xut;rowNames=["mean","variance","std"])
```

mean 0.098
variance 0.005
std 0.074

Linear Regressions

using the included function `OLSGMFn(Y,X)`

```

x = readlm("Data/FFmFactorsPs.csv",',',skipstart=1)

#yearmonth, market, small minus big, high minus low
(ym,Rme,RSMB,RHML) = (x[:,1],x[:,2]/100,x[:,3]/100,x[:,4]/100)
x = nothing

printlnPs("Sample size:",size(Rme))

```

Sample size: (660,)

```

Y = Rme #T-vector, to get standard OLS notation
T = size(Y,1)
X = [ones(T) RSMB RHML] #TxK matrix, regressors

(b,-,V,R²) = OlsGM(Y,X)
Stdb = sqrt.(diag(V)) #standard errors, assuming iid errors, diag is from `LinearAlgebra`

printblue("OLS Results:\n")
xNames = ["c","SMB","HML"]
printmat(b,Stdb,colNames=["b","std"],rowNames=xNames)

printlnPs("R²: ",R²)

```

OLS Results:

	b	std
c	0.006	0.002
SMB	0.380	0.056
HML	-0.257	0.055

R²: 0.108

Test a Single Coefficient

Suppose we want to test if the 2nd coefficient is equal to 0.3. This can be done as follows. If the |t-stat|, is larger than 1.645, then you cannot reject the null hypothesis on the 10% significance level

```

t = (b[2]-0.3)/Stdb[2]

printlnPs("|t| : ",abs(t))

```

$|t|$: 1.431

Optimization

of linear-quadratic problems, using the [OSQP.jl](#) package. An alternative, [Clarabel.jl](#) is discussed briefly at the end of the notebook.

The methods illustrated here are well suited for cases when the objective is linear-quadratic, for instance, the portfolio variance ($w'\Sigma w$) or when the estimation problem is based on minimizing the sum of squared residuals ($u'u$), and the restrictions are linear expressions.

Load Packages and Utility Functions

```
using Printf, LinearAlgebra, SparseArrays, OSQP

include("src/printmat.jl");
```

```
using Plots, LaTeXStrings      #LaTeXStrings to facilitate using LaTeX in plots
default(size = (480,320),fmt = :png)
```

The OSQP.jl Optimization Package

The [OSQP.jl](#) package is tailor made for solving linear-quadratic problems (with linear restrictions). It solves problems of the type

$$\min 0.5\theta'P\theta + q'\theta \text{ subject to } l \leq A\theta \leq u,$$

where θ is a vector of choice variables.

To get an equality restriction in row i , set $l[i]=u[i]$.

Notice that (P,A) should be Sparse matrices and (q,l,u) vectors with Float64 numbers.

We consider several cases below: no restrictions on θ , bounds on θ , and a linear equality restriction.

Unconstrained Minimization

We will define the following minimization problem

$$P = 2 \begin{bmatrix} 1 & 0 \\ 0 & 16 \end{bmatrix}$$

and

$$q = \begin{bmatrix} -4 \\ 24 \end{bmatrix}.$$

It is straightforward to see that this is the same as minimizing $(\theta_1 - 2)^2 + (4\theta_2 + 3)^2$

If there are no constraints, then the solution should be $\theta = (2, -3/4)$.

Before doing the optimization, we plot the contours of the loss function.

```
P = 2*[1 0;      #we minimize 0.5*theta'P*theta + q'*theta, hence the 2*[]
        0 16]    #this is the same as minimizing (x-2)^2 + (4y+3)^2
q = [-4.0, 24.0] #vector, Float64

function Lossfn(P,q,theta)
    L = 0.5*theta'*P*theta + q'*theta
    return L
end

(n1,n2) = (82,122)
theta1 = range(1,5,length=n1)
theta2 = range(-1,0,length=n2)

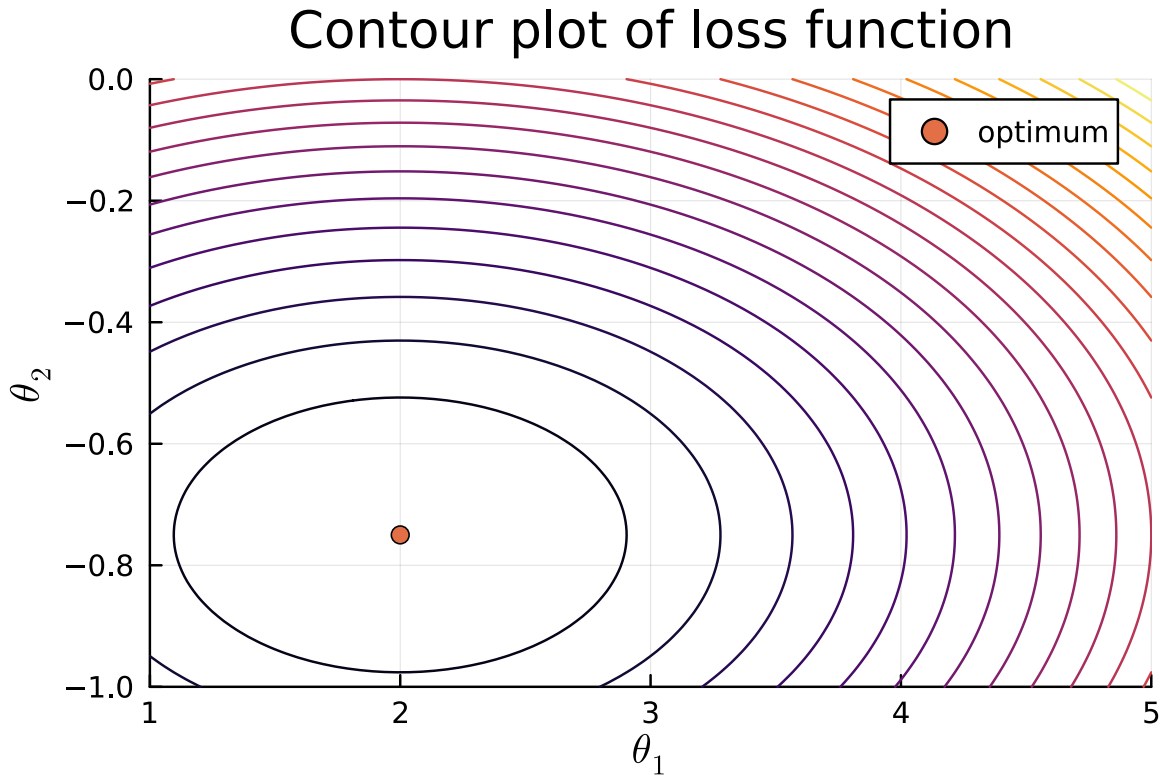
loss2d = fill{NaN}(n1,n2) #matrix with loss fn values
for i in 1:n1, j in 1:n2
    loss2d[i,j] = Lossfn(P,q,[theta1[i],theta2[j]])
end

p1 = contour(theta1,theta2,loss2d', #notice: loss2d'
             xlims = (1,5),
             ylims = (-1,0),
             legend = false,
             levels = 21,
             title = "Contour plot of loss function",
             xlabel = L"\theta_1",
```

```

ylabel = L"\theta_2" )
scatter!([2],[-0.75],label="optimum",legend=true)
display(p1)

```



```

A = zeros(1,2)          #effectively no restriction
l = [0.0]               #vectors, Float64
u = [0.0]

settings = Dict{:verbose => true}          #to print more info on iterations
model = OSQP.Model()                      #create model
OSQP.setup!(model; P=sparse(P), q=q, A=sparse(A), l=l, u=u, settings...)  #set up model, P and A s
result = OSQP.solve!(model)               #solve

printblue("Unconstrained minimization: the solution should be (2,-3/4):")
printmat(result.x)
printblue("\nand lots of information about the iterations:")

```

Unconstrained minimization: the solution should be (2,-3/4):

2.000
-0.750

and lots of information about the iterations:

```
-----  
OSQP v0.6.2 - Operator Splitting QP Solver  
  (c) Bartolomeo Stellato, Goran Banjac  
University of Oxford - Stanford University 2021  
-----  
problem: variables n = 2, constraints m = 1  
         nnz(P) + nnz(A) = 2  
settings: linear system solver = qdldl,  
         eps_abs = 1.0e-003, eps_rel = 1.0e-003,  
         eps_prim_inf = 1.0e-004, eps_dual_inf = 1.0e-004,  
         rho = 1.00e-001 (adaptive),  
         sigma = 1.00e-006, alpha = 1.60, max_iter = 4000  
         check_termination: on (interval 25),  
         scaling: on, scaled_termination: off  
         warm start: on, polish: off, time_limit: off  
  
iter  objective    pri res    dua res    rho        time  
   1  -8.3200e+000  0.00e+000  1.44e+001  1.00e-001  7.10e-005s  
  25 -1.3000e+001  0.00e+000  6.82e-005  1.00e-001  7.75e-005s  
  
status:                solved  
number of iterations: 25  
optimal objective:     -13.0000  
run time:               8.03e-005s  
optimal rho estimate: 1.00e-006
```

Constrained Minimization

Bounds on the solution: $2.75 \leq \theta_1$ and $\theta_2 \leq -0.3$.

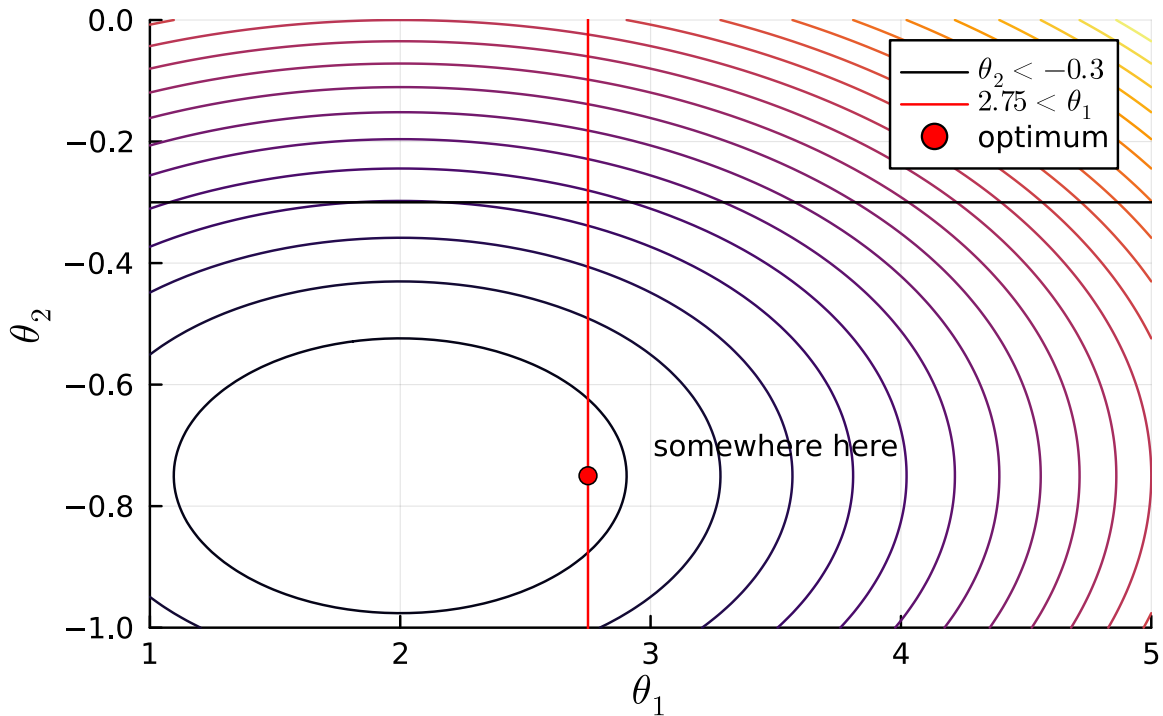
```
p1 = contour(  $\theta_1, \theta_2$ , loss2d',  
             xlims = (1,5),  
             ylims = (-1,0),  
             legend = false,  
             levels = 21,  
             title = "Contour plot of loss function",  
             xlabel = L"\theta_1",
```

```

        ylabel = L"\theta_2",
        annotation = (3.5,-0.7,text("somewhere here",8)) )
hline!([-0.3],linecolor=:black,label=L"\theta_2 < -0.3")
vline!([2.75],linecolor=:red,line=:solid,label=L"2.75 < \theta_1")
scatter!([2.75],[-0.75],color=:red,label="optimum",legend = true)
display(p1)

```

Contour plot of loss function



```

A = I[1:2,1:2]      #identity matrix, I2
l = [2.75,-Inf]      #lower bounds on A*x
u = [Inf,-0.3]       #upper bounds, 2.75 <= θ1 <= Inf, -Inf <= θ2 <= -0.3

settings = Dict(:verbose => false)
model = OSQP.Model()
OSQP.setup!(model; P=sparse(P), q=q, A=sparse(A), l=l, u=u, settings...)
result = OSQP.solve!(model)

printblue("with bounds on the solution: the solution should be (2.75,-0.75):")
printmat(result.x)

```

```
printblue("checking if the bounds are satisfied:")
printmat(l,A*result.x,u;colNames=["l","Aθ","u"])
```

with bounds on the solution: the solution should be (2.75,-0.75):

```
2.750
-0.750
```

checking if the bounds are satisfied:

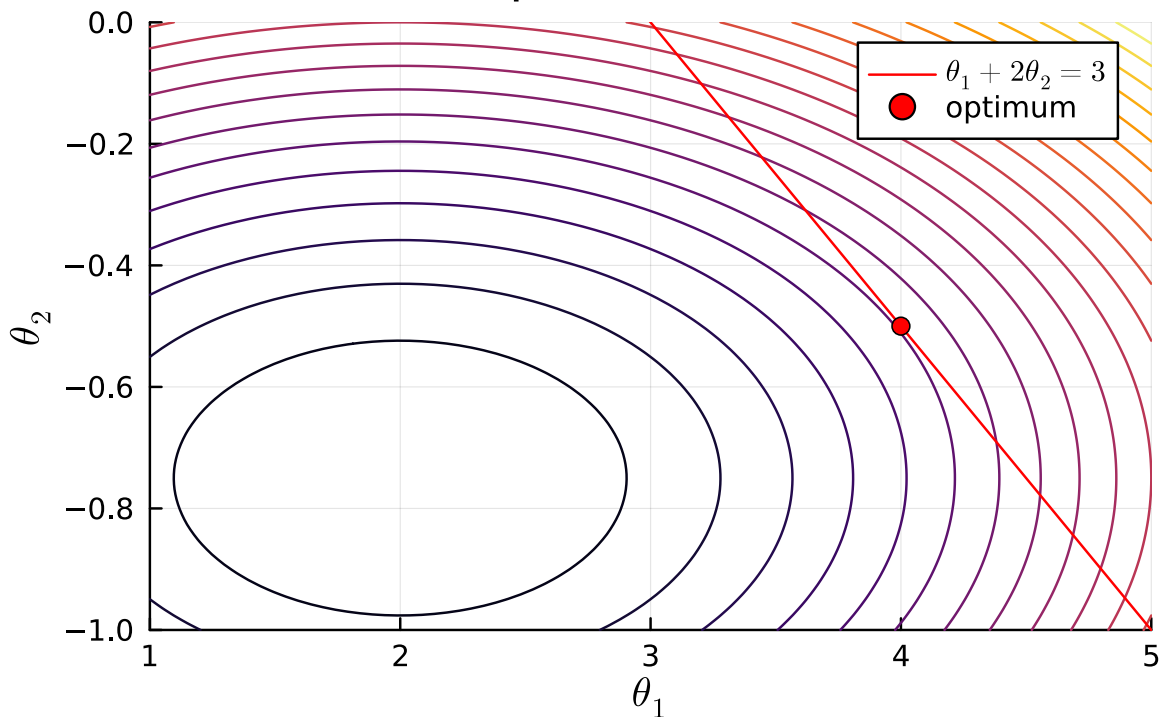
l	Aθ	u
2.750	2.750	Inf
-Inf	-0.750	-0.300

Constrained Minimization

A linear equality constraint: $\theta_1 + 2\theta_2 = 3$.

```
p1 = contour(θ1,θ2,loss2d',
             xlims = (1,5),
             ylims = (-1,0),
             legend = false,
             levels = 21,
             title = "Contour plot of loss function",
             xlabel = L"\theta_1",
             ylabel = L"\theta_2" )
Plots.abline!(-0.5,1.5,linecolor=:red,line=:solid,label=L"\theta_1+2\theta_2=3")
scatter!([4],[-0.5],markercolor=:red,label="optimum",legend=true)
display(p1)
```

Contour plot of loss function



```
A = [1 2]                                #equality constraint
l = [3.0]
u = [3.0]

model = OSQP.Model()
OSQP.setup!(model; P=sparse(P), q=q, A=sparse(A), l=l, u=u, settings...)
result = OSQP.solve!(model)

printblue("equality constraint: the solution should be (4,-1/2):\n")
printmat(result.x)
printblue("checking if the bounds are satisfied:")
printmat(l,A*result.x,u;colNames=["l","Aθ","u"])
```

equality constraint: the solution should be (4,-1/2):

```
4.000
-0.500
```

checking if the bounds are satisfied:

l	$A\theta$	u
3.000	3.000	3.000

Updating the Problem (extra)

When you just want to change some of the inputs to the optimization problem, then it may pay off to do `update!()`. This is especially useful when you resolve the problem many times (in a loop, say).

```
model = OSQP.Model()           #setting up the problem for the first time
OSQP.setup!(model; P=sparse(P), q=q, A=sparse(A), l=l, u=u, settings...)
result = OSQP.solve!(model)
printmat(result.x)

q_new = [-2.0, 20.0]           #resolve the problem with different q values
OSQP.update!(model; q=q_new)   #(partially) update the problem
result = OSQP.solve!(model)
printmat(result.x)
```

```
4.000
-0.500
```

```
3.600
-0.300
```

An Alternative Package (extra)

The [Clarabel.jl](#) package is an interesting alternative to OSQP.jl. It has a slightly different syntax, but the `DoClarabel()` function in `src/DoClarabel.jl` provides a way to call it using the same syntax as for OSQP.jl.

```
using LinearAlgebra, SparseArrays, Clarabel
include("src/DoClarabel.jl");
```

```
(x,result,fn0) = DoClarabel(P,q,A,l,u,Clarabel.Settings(verbose = false))
printblue("equality constraint: the solution should be (4,-1/2), as before:")
printmat(x)
```

equality constraint: the solution should be $(4, -1/2)$, as before:

4.000

-0.500

Numerical Optimization

This notebook uses the [Optim.jl](#) package for unconstrained and box constrained (lower and upper bounds).

For other types of constraints, consider [Optimization.jl](#), [NLOpt.jl](#), or [JuMP.jl](#).

The optimization problems are (for pedagogical reasons) the same as in the other notebook about optimization. This means that the solutions should be very similar and that the contour plots in the other notebook can be used as references.

However, the current notebook is focused on methods for solving general optimization problems. In contrast, the other notebook focuses on linear-quadratic problems (mean-variance, least squares, etc), where there are faster algorithms.

Load Packages and Extra Functions

```
using Printf, Optim

include("src/printmat.jl");
```

```
using Plots
default(size = (480,320),fmt = :png)
```

Optimization with One Choice Variable

Running

```
Sol = optimize(x->fn1(x,0.5),a,b)
```

finds the x value (in the interval $[a,b]$) that *minimizes* $fn1(x,0.5)$. The $x \rightarrow fn1(x,0.5)$ syntax makes this a function of x only, which is what the `optimize()` function wants.

The `optimize()` is here called without specifying any options. This means that defaults will be used, for instance, for the algorithm choice.

The output (Sol) contains a lot of information.

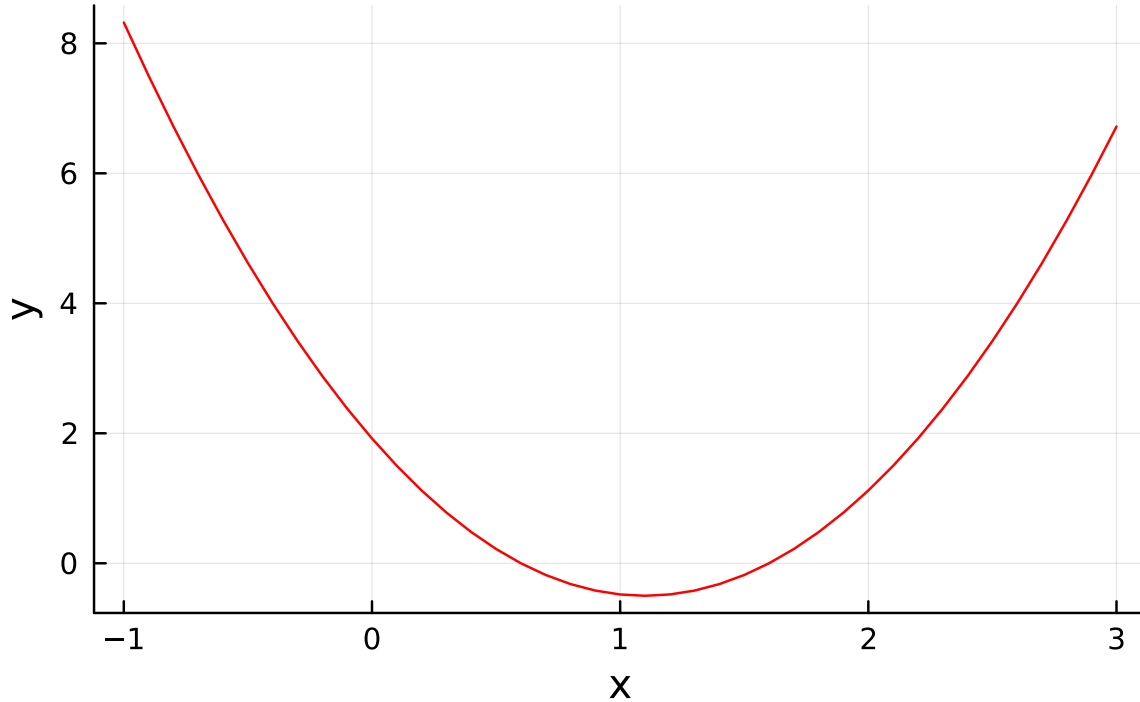
```
function fn1(x,c)                                #notice: the function has two arguments
    value = 2*(x - 1.1)^2 - c
    return value
end
```

fn1 (generic function with 1 method)

```
x = -1:0.1:3

p1 = plot( x,fn1.(x,0.5),                        #plotting the function can help identify
           linecolor = :red,                    #starting points and possible issues
           linewidth = 1,
           legend = nothing,
           title = "the fn1(x,0.5) function",
           xlabel = "x",
           ylabel = "y" )
display(p1)
```


the fn1(x,0.5) function



```
Sol = optimize(x→fn1(x,0.5),-2.0,3.0) #using a default algorithm
println(Sol)

printblue("\nThe minimum is at: ", Optim.minimizer(Sol)) #the optimal x value
printblue("Compare with the plot above")
```

Results of Optimization Algorithm

```
* Status: success
* Algorithm: Brent's Method
* Search Interval: [-2.000000, 3.000000]
* Minimizer: 1.100000e+00
* Minimum: -5.000000e-01
* Iterations: 5
* Convergence: max(|x - x_upper|, |x - x_lower|) <= 2*(1.5e-08*|x|+2.2e-16): true
* Objective Function Calls: 6
```

The minimum is at: 1.1

Compare with the plot above

One Choice Variable: Supplying a Starting Guess Instead (extra)

If you prefer to give a starting guess x_0 instead of an interval, then supply it as a vector $[x_0]$. We also use `x→fn1(only(x),0.05)` to make sure that the `fn1()` function gets a scalar, not a 1-element vector. The solution is then also a vector (with one element.)

```
x0 = 1.0
Solb = optimize(x→fn1(only(x),0.5),[x0])

printlnPs("The minimum is at: ",Optim.minimizer(Solb))
```

The minimum is at: 1.100

Several Choice Variables: Unconstrained Optimization

In the example below, we choose (x,y) so as to minimize the fairly simple objective function

$$(x + c_1)^2 + (4y + c_2)^2,$$

without any constraints. We will use $(c_1, c_2) = (-2, 3)$ in the example below, so the solution should be $(x,y) = (2, -3/4)$.

The `BFGS()` algorithm is good (and quick) for smooth problems. Otherwise, use the default (Nelder-Mead()). Unless we specify a function (or algorithm) for the gradient, `optimize()` will calculate it numerically. This typically works well for problems with a small/moderate number of choice variables.

```
function fn2(θ,c)
    (x,y) = (θ[1],θ[2])          #unpack the choice variables and get nicer names
    L     = (x+c[1])^2 + (4*y+c[2])^2
    return L
end
```

fn2 (generic function with 1 method)

```
c = [-2,3]
θ0 = [4.0,-0.5]                #starting guess

Sol = optimize(u→fn2(u,c),θ0,BFGS()) #BFGS() is one of several algorithm choices

printlnPs("minimum at (x,y)= ",Optim.minimizer(Sol))
```

minimum at (x,y)= 2.000 -0.750

Several Choice Variables: Bounds on the Solutions

The next few cells discuss how to impose bounds on the solution.

In the example below, we impose $2.75 \leq x$ (a lower bound) and $y \leq -0.3$ (an upper bound). We will see that only one of these restrictions binds, as the solution should be $(x, y) = (2.75, -3/4)$.

We here use `Fminbox(BFGS())` to use `BFGS()` but let `optimize()` make sure that the bounds are respected.

```
lower_θ = [2.75, -Inf]
upper_θ = [Inf, -0.3]

Sol = optimize(u→fn2(u,c),lower_θ,upper_θ,θ₀,Fminbox(BFGS()))
printlnPs("The optimum is at (x,y) = ",Optim.minimizer(Sol))
```

The optimum is at (x,y) = 2.750 -0.750

Several Choice Variables: Supplying the Gradient (extra)

Supplying a function for calculating the derivatives improves speed and accuracy. See below for an example.

```
function g2!(G,θ,c)                                #derivatives of fn2 wrt. θ[1] and θ[2]
    (x,y) = (θ[1],θ[2])
    G .= (2*(x+c[1]), 2*4*(4*y+c[2]))              #the tuple () fills/updates an existing G vector, creates new if not
    return G
end

Sol3 = optimize(u→fn2(u,c),(G,u)→g2!(G,u,c),[1.0,-0.5],BFGS())
println(Sol3)
printblue("Probably faster than the solution above")
printblue("Minimum at: ",round.(Optim.minimizer(Sol3),digits=3))
```

* Status: success

* Candidate solution

Final objective value: 0.000000e+00

* Found with

Algorithm: BFGS

* Convergence measures

$ x - x' $	$= 9.34e-01 \not\leq 0.0e+00$
$ x - x' / x' $	$= 4.67e-01 \not\leq 0.0e+00$
$ f(x) - f(x') $	$= 8.75e-01 \not\leq 0.0e+00$
$ f(x) - f(x') / f(x') $	$= \text{Inf} \not\leq 0.0e+00$
$ g(x) $	$= 0.00e+00 \leq 1.0e-08$

* Work counters

Seconds run:	0 (vs limit Inf)
Iterations:	2
f(x) calls:	5
$\nabla f(x)$ calls:	5
$\nabla f(x)^T v$ calls:	0

Probably faster than the solution above

Minimum at: [2.0, -0.75]